

# BIG DATA

## DIPLOMADO DE DATOS 2026

### Clase 7: NoSQL (I)

Alberto Moya

[albertomoya87@gmail.com](mailto:albertomoya87@gmail.com)

DATOS MASIVOS:

ALMACENAR DATOS ESTRUCTURADOS

# Bases de datos relacionales



# Bases de datos relacionales: ¿Talla única?

## “One Size Fits All”: An Idea Whose Time Has Come and Gone

Michael Stonebraker  
*Computer Science and Artificial  
Intelligence Laboratory, M.I.T., and  
StreamBase Systems, Inc.*  
stonebraker@csail.mit.edu

Uğur Çetintemel  
*Department of Computer Science  
Brown University, and  
StreamBase Systems, Inc.*  
ugur@cs.brown.edu

### Abstract

*The last 25 years of commercial DBMS development can be summed up in a single phrase: “One size fits all”. This phrase refers to the fact that the traditional DBMS architecture (originally designed and optimized for business data processing) has been used to support many data-centric applications with widely varying characteristics and requirements.*

*In this paper, we argue that this concept is no longer applicable to the database market, and that the commercial world will fracture into a collection of independent database engines, some of which may be unified by a common front-end parser. We use examples from the stream-processing market and the data-warehouse market to bolster our claims. We also briefly discuss other markets for which the traditional architecture is a poor fit and argue for a critical rethinking of the current factoring of systems services into products.*

of multiple code lines causes various practical problems, including:

- *a cost problem*, because maintenance costs increase at least linearly with the number of code lines;
- *a compatibility problem*, because all applications have to run against every code line;
- *a sales problem*, because salespeople get confused about which product to try to sell to a customer; and
- *a marketing problem*, because multiple code lines need to be positioned correctly in the marketplace.

To avoid these problems, all the major DBMS vendors have followed the adage “put all wood behind one arrowhead”. In this paper we argue that this strategy has failed already, and will fail more dramatically off into the future.

The rest of the paper is structured as follows. In Section 2, we briefly indicate why the single code-line strategy has failed already by citing some of the key characteristics of the data warehouse market. In Section

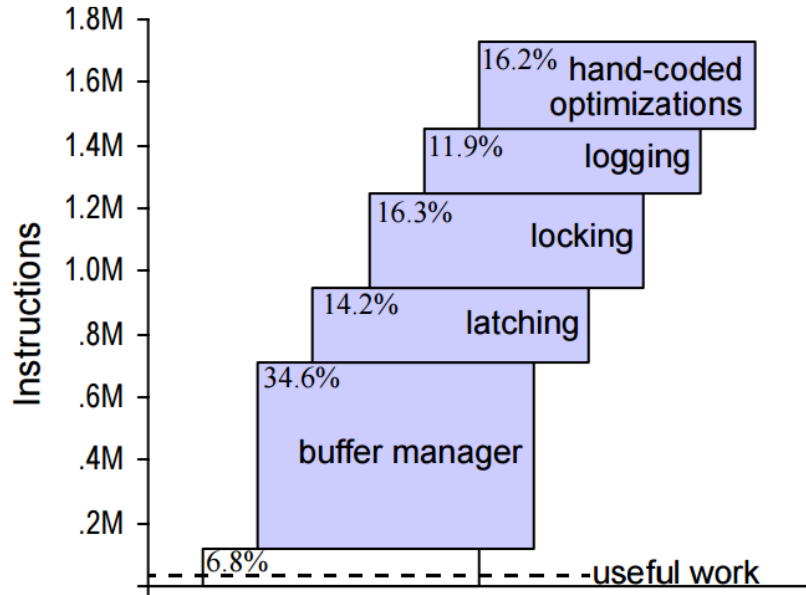




# Bases de datos relacionales: Aspectos costosos

- "Structured Query Language" (SQL):
  - Lenguaje declarativo
  - Muchas buenas características
  - **Difícil de optimizar**
- "Atomicity, Consistency, Isolation, Durability" (ACID):
  - Asegura correctitud de la base de datos
    - ¡Incluso si hay mucho tráfico!
  - **¡Las transacciones incurren en altos gastos!**
    - bloqueos multi-fase, multi-versionamiento, logging
- La distribución no es directa

# Gastos en transacciones: el costo de ACID



- 640 transacciones por segundo en un sistema que garantiza ACID
- 12.700 transacciones por segundo en un sistema sin logging, transacciones o bloqueos

## OLTP Through the Looking Glass, and What We Found There

Stavros Harizopoulos  
HP Labs  
Palo Alto, CA  
stavros@hp.com

Daniel J. Abadi  
Yale University  
New Haven, CT  
dna@cs.yale.edu

Samuel Madden Michael Stonebraker  
Massachusetts Institute of Technology  
Cambridge, MA  
{madden, stonebraker}@csail.mit.edu

### ABSTRACT

Online Transaction Processing (OLTP) databases include a suite of features — disk-resident B-trees and heap files, locking-based concurrency control, support for multi-threading — that were optimized for computer technology of the late 1970's. Advances in modern processors, memories, and networks mean that today's computers are vastly different from those of 30 years ago, such that many OLTP databases will now fit in main memory, and most OLTP transactions can be processed in milliseconds or less. Yet database architecture has changed little.

### 1. INTRODUCTION

Modern general purpose online transaction processing (OLTP) database systems include a standard suite of features: a collection of on-disk data structures for table storage, including heap files and B-trees, support for multiple concurrent queries via locking-based concurrency control, log-based recovery, and an efficient buffer manager. These features were developed to support transaction processing in the 1970's and 1980's, when an OLTP database was many times larger than the main memory, and when the computers that ran these databases cost hundreds of thousands to

¿ALTERNATIVAS A BASES DE DATOS RELACIONALES  
PARA GESTIONAR DATOS MASIVOS?



# NoSQL

¿Qué saben ustedes de NoSQL?

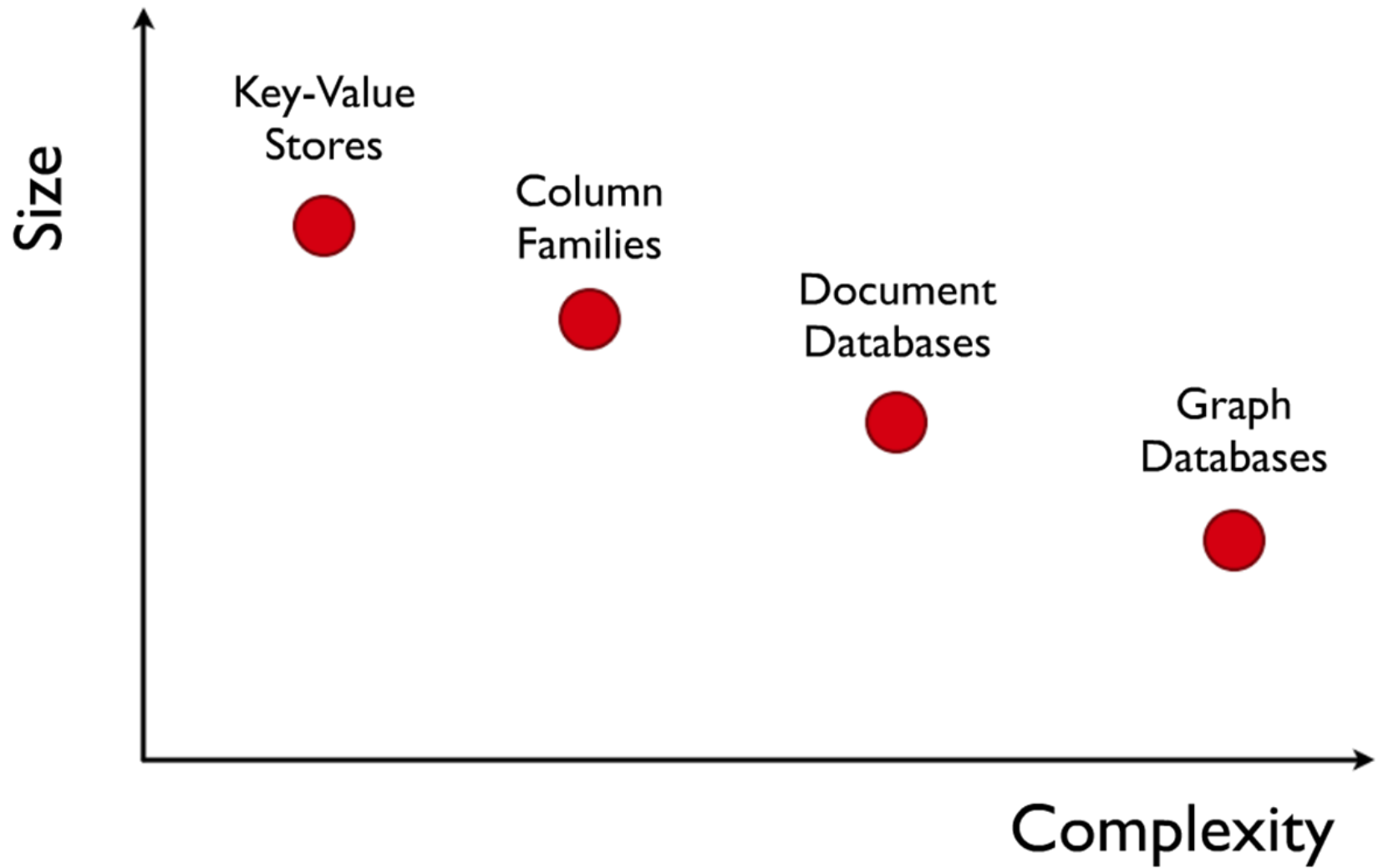


Not  
Only SQL



| Rank     |          |          | DBMS                         | Database Model             | Score    |          |          |
|----------|----------|----------|------------------------------|----------------------------|----------|----------|----------|
| Jul 2025 | Jun 2025 | Jul 2024 |                              |                            | Jul 2025 | Jun 2025 | Jul 2024 |
| 1.       | 1.       | 1.       | Oracle                       | Relational, Multi-model ⓘ  | 1217.05  | -13.33   | -23.31   |
| 2.       | 2.       | 2.       | MySQL                        | Relational, Multi-model ⓘ  | 940.73   | -12.85   | -98.73   |
| 3.       | 3.       | 3.       | Microsoft SQL Server         | Relational, Multi-model ⓘ  | 771.14   | -5.61    | -36.51   |
| 4.       | 4.       | 4.       | PostgreSQL                   | Relational, Multi-model ⓘ  | 680.89   | +0.23    | +41.98   |
| 5.       | 5.       | 5.       | MongoDB 📦                    | Document, Multi-model ⓘ    | 403.83   | +0.99    | -25.99   |
| 6.       | 6.       | ↑ 7.     | Snowflake                    | Relational                 | 176.17   | +1.68    | +39.64   |
| 7.       | 7.       | ↓ 6.     | Redis                        | Key-value, Multi-model ⓘ   | 149.72   | -2.01    | -7.05    |
| 8.       | 8.       | ↑ 9.     | IBM Db2                      | Relational, Multi-model ⓘ  | 127.51   | +2.38    | +3.11    |
| 9.       | 9.       | ↓ 8.     | Elasticsearch                | Multi-model ⓘ              | 118.83   | -2.45    | -12.00   |
| 10.      | 10.      | 10.      | SQLite                       | Relational                 | 115.44   | -1.60    | +5.49    |
| 11.      | 11.      | ↑ 12.    | Apache Cassandra             | Wide column, Multi-model ⓘ | 108.76   | +0.49    | +9.63    |
| 12.      | 12.      | ↑ 15.    | Databricks                   | Multi-model ⓘ              | 108.04   | +3.36    | +24.74   |
| 13.      | 13.      | ↑ 14.    | MariaDB 📦                    | Relational, Multi-model ⓘ  | 95.44    | +0.91    | +4.86    |
| 14.      | 14.      | ↓ 11.    | Microsoft Access             | Relational                 | 90.47    | +2.19    | -10.17   |
| 15.      | 15.      | ↑ 17.    | Amazon DynamoDB              | Multi-model ⓘ              | 84.15    | +0.81    | +13.20   |
| 16.      | ↑ 17.    | 16.      | Microsoft Azure SQL Database | Relational, Multi-model ⓘ  | 76.51    | +1.20    | -0.24    |
| 17.      | ↓ 16.    | ↑ 19.    | Apache Hive                  | Relational                 | 75.74    | -0.94    | +18.45   |
| 18.      | 18.      | ↓ 13.    | Splunk                       | Search engine              | 69.52    | -0.10    | -23.40   |
| 19.      | 19.      | ↓ 18.    | Google BigQuery              | Relational                 | 64.19    | -0.35    | +6.38    |
| 20.      | 20.      | ↑ 21.    | Neo4j                        | Graph                      | 54.64    | +3.30    | +8.88    |

# NoSQL



# NoSQL: No solo SQL ("Not only SQL")

- **Distribuido**
  - "Sharding": partición "horizontal" de datos
  - Replicación
  - Garantías diferentes a ACID
- A menudo **lenguajes más simples** que SQL
  - APIs no estandarizadas
  - Más trabajo para la aplicación
- **Sabores diferentes** (para escenarios diferentes)
  - Garantías diferentes
  - Perfiles diferentes de escalabilidad
  - Funcionalidades de consulta diferentes
  - Modelos de datos diferentes

LIMITACIONES DE ALMACENAMIENTO  
DISTRIBUIDO: TEOREMA CAP

# Pero primero ... ACID

Para bases de datos tradicionales sin distribución ...

## 1. Atomicity:

- Atomicidad: Las transacciones son todo o nada

## 2. Consistency:

- Coherencia: No rompe las restricciones / reglas

## 3. Isolation:

- Aislamiento: Transacciones paralelas actúan como si fueran secuenciales

## 4. Durability

- Durabilidad: El sistema recuerda los cambios



# ¿Qué es CAP?

Tres garantías que un sistema distribuido podría ofrecer

## 1. Consistency:

- Coherencia: Todos los nodos tienen una visión coherente del sistema

## 2. Availability:

- Disponibilidad: Toda petición de escritura/lectura es atendida

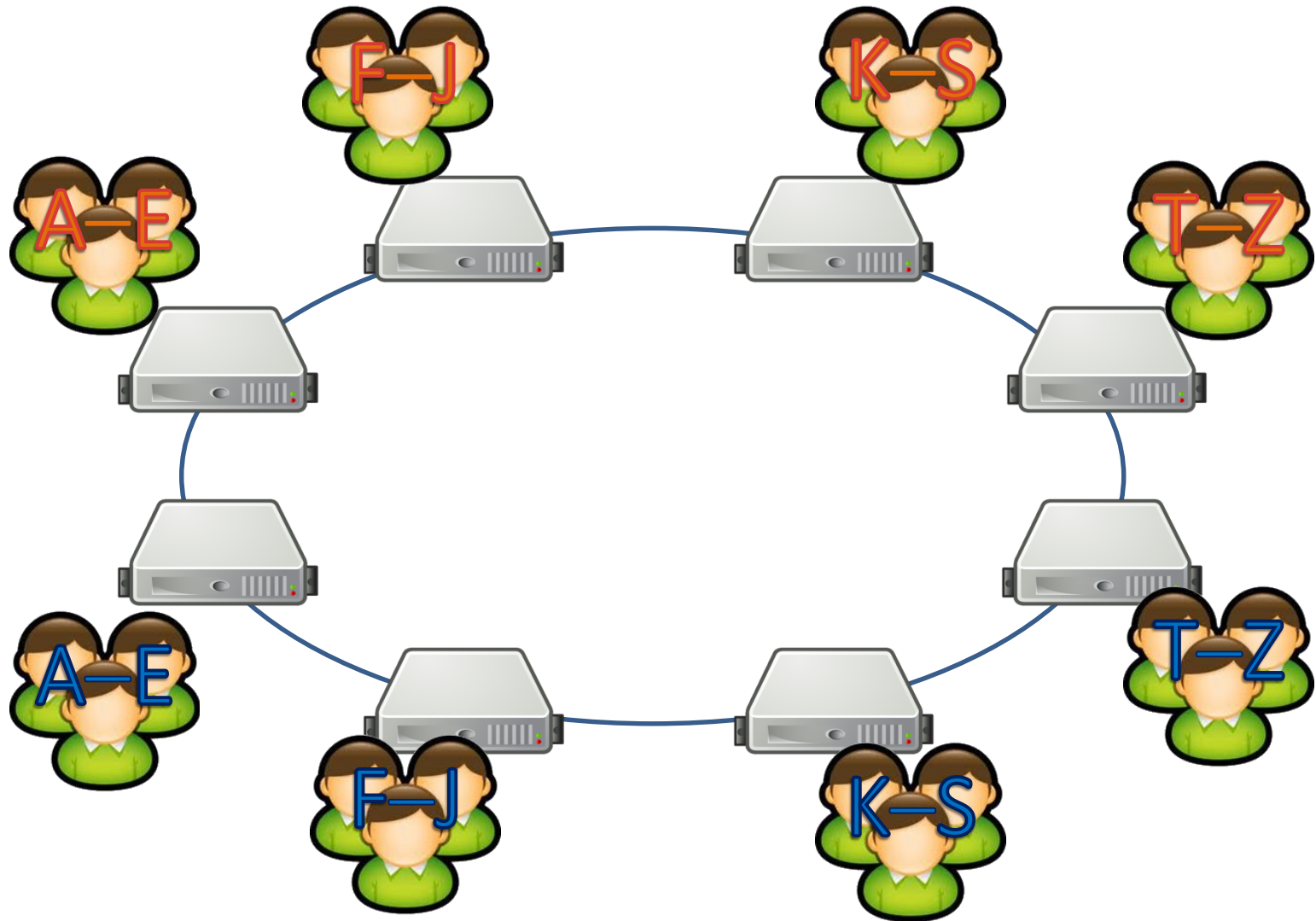
## 3. Partition-tolerance:

- Tolerancia a particiones: El sistema funciona incluso si hay pérdida de mensajes

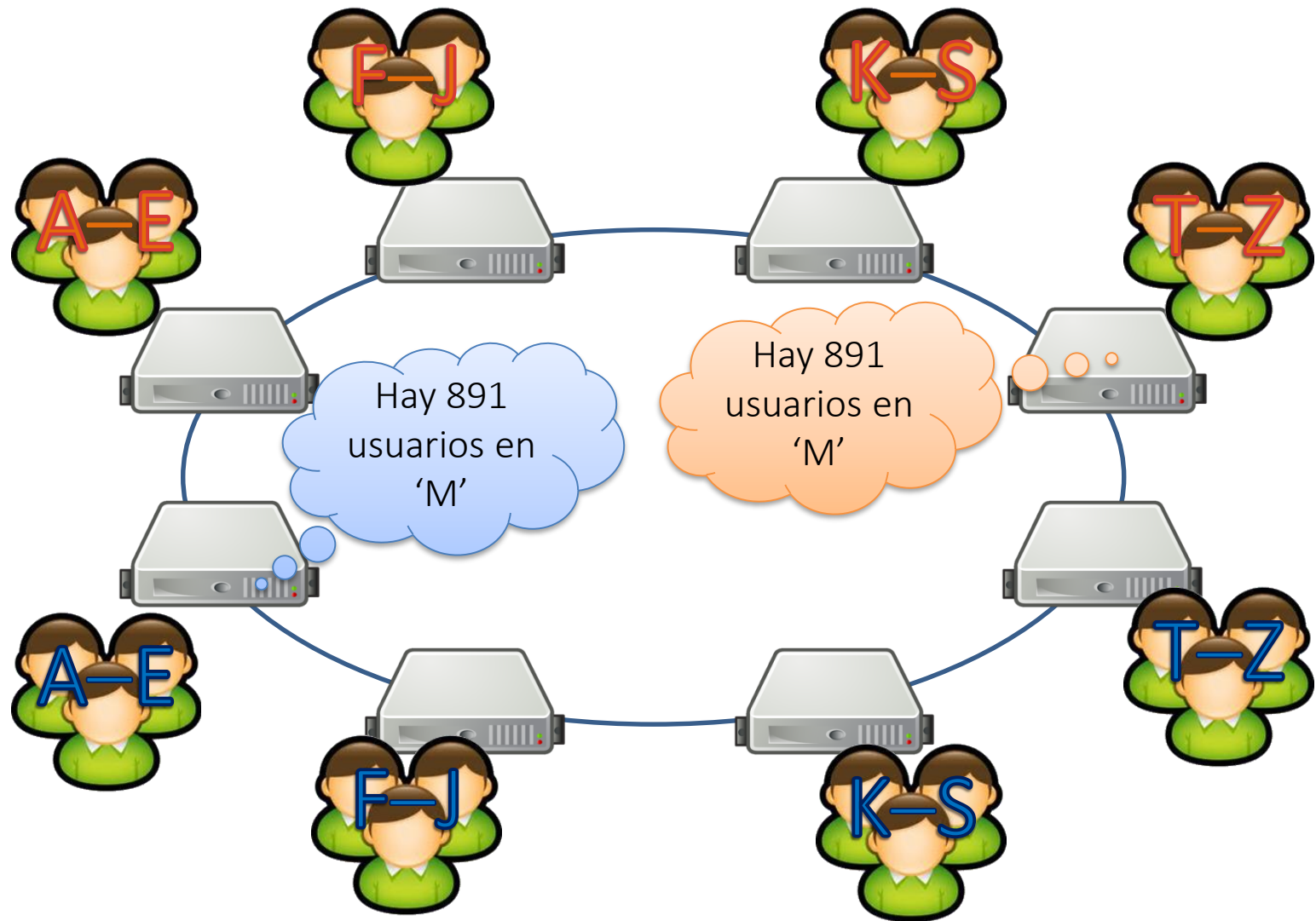
¡El CA en CAP no es el mismo CA que en ACID! 

¡Partición de la red, no de los datos! 

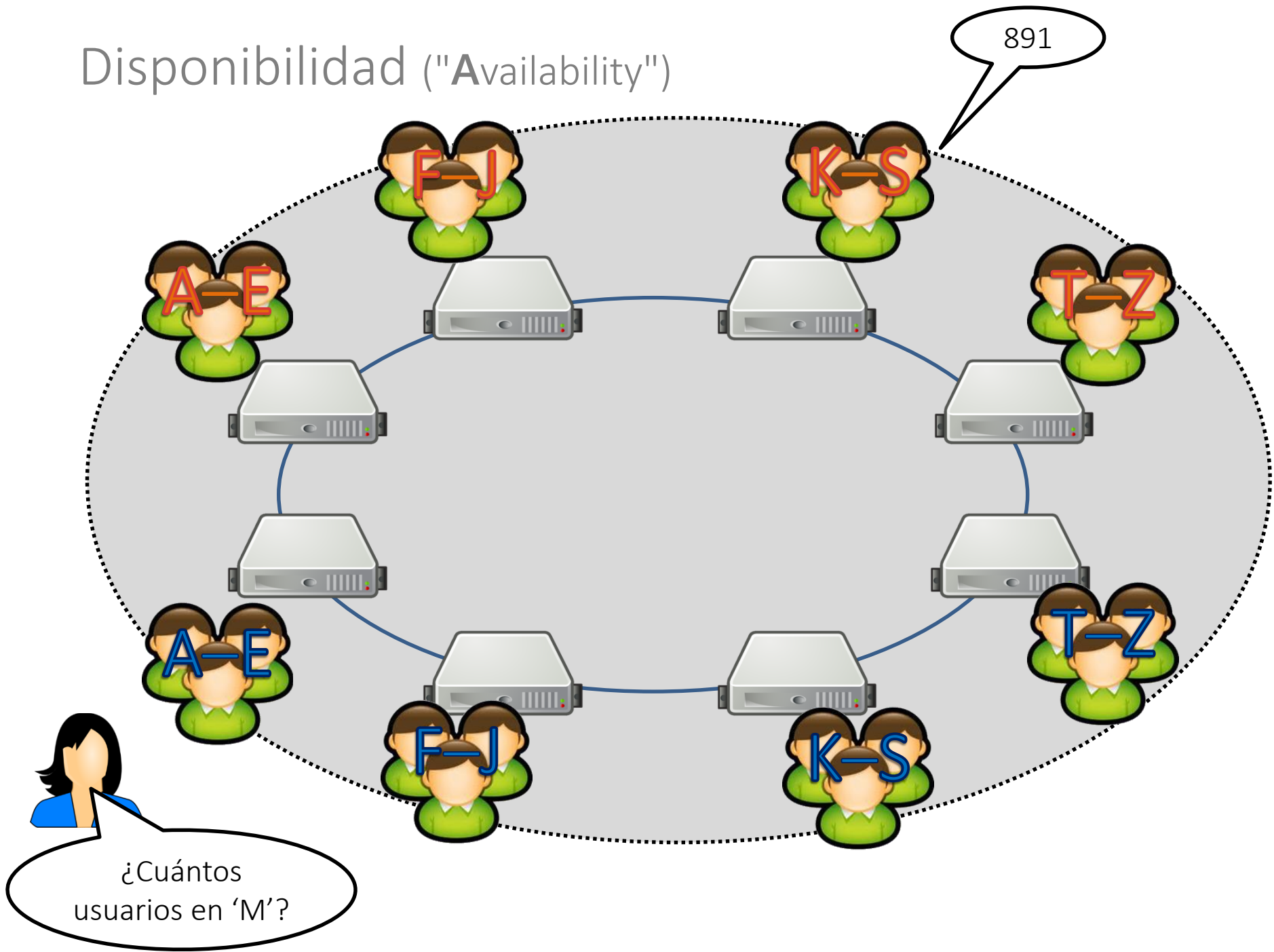
# Un sistema distribuido (con Replicación)



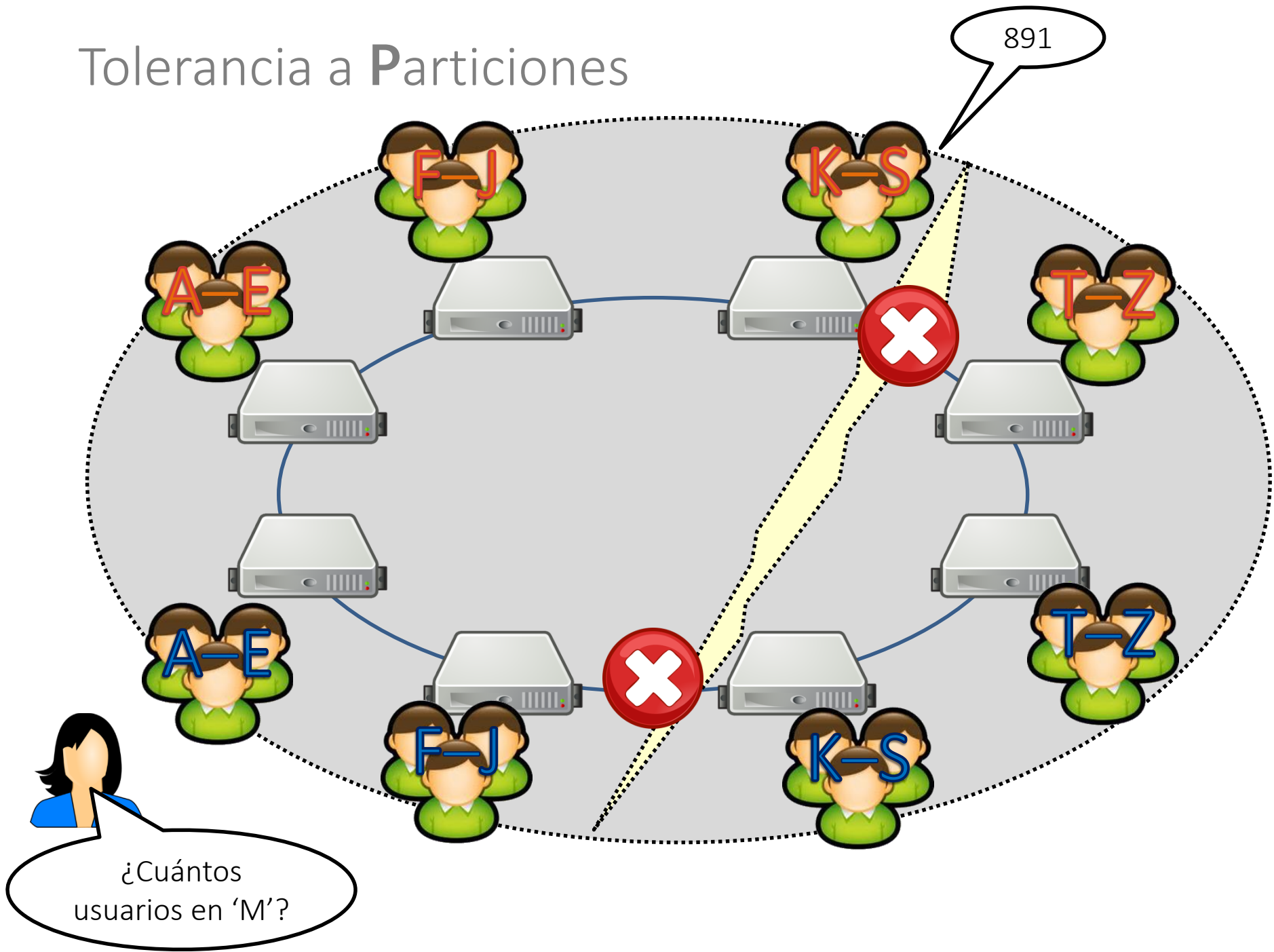
# Coherencia



# Disponibilidad ("Availability")



# Tolerancia a Particiones



# La pregunta CAP

¿Puede un sistema distribuido garantizar:

**Coherencia** (todos los nodos tienen una visión actualizada),

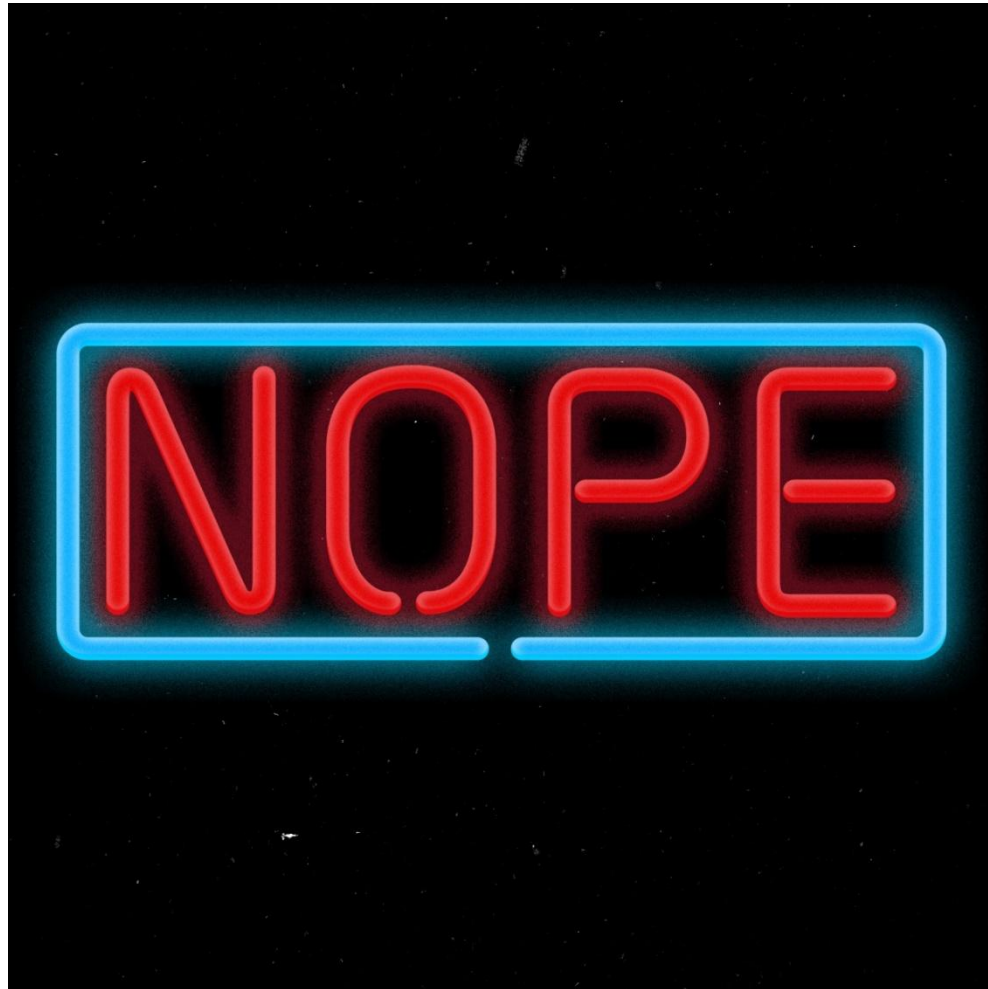
**disponibilidad/Availability** (toda escritura/lectura es atendida) y

**tolerancia a Particiones** (el sistema funciona incluso si hay mensajes perdidos)  
al mismo tiempo?

¿Qué opinas?



La respuesta CAP



# El teorema CAP

¡Un sistema distribuido no puede garantizar:

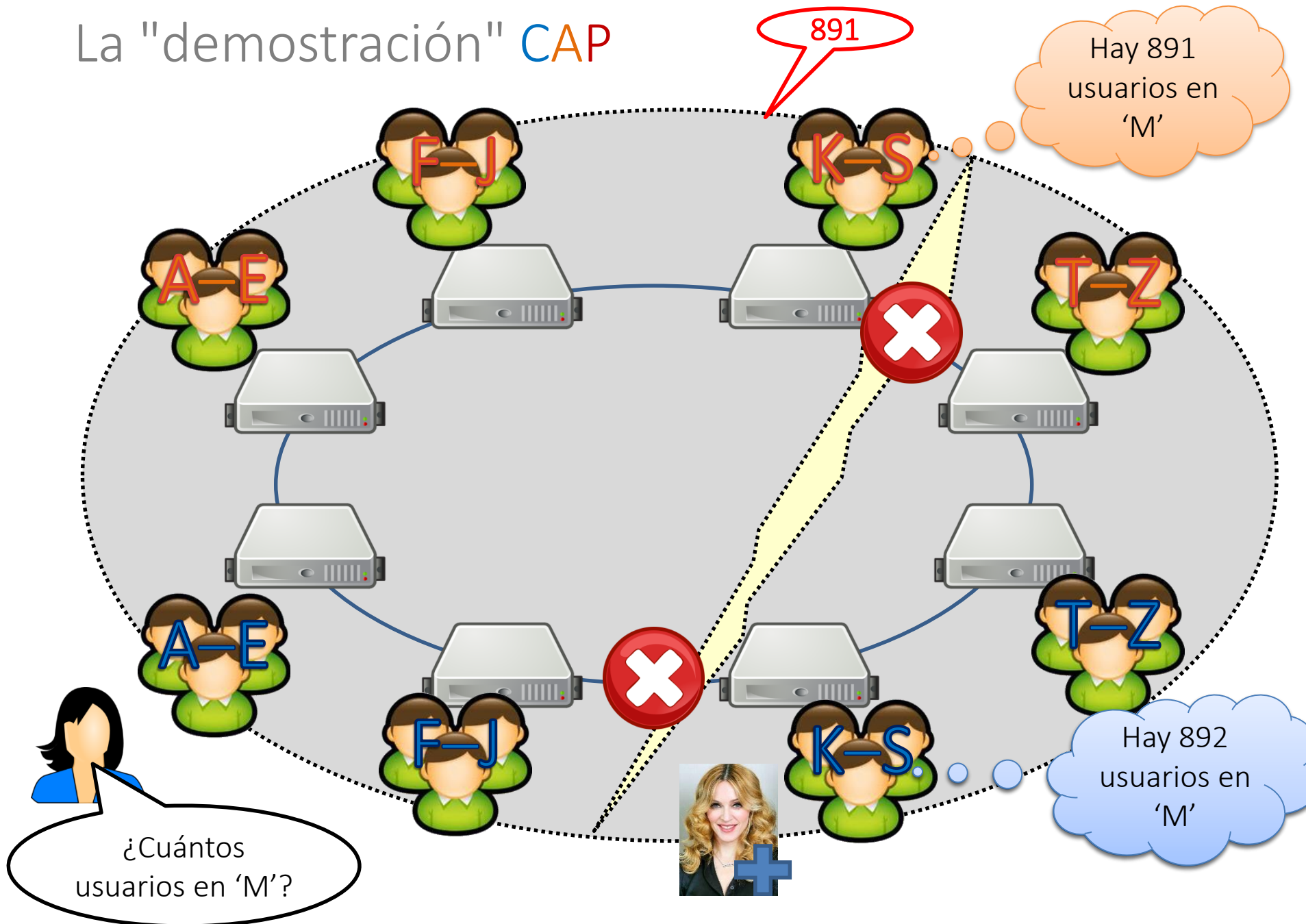
**Coherencia** (todos los nodos tienen una visión actualizada),

**disponibilidad/Availability** (toda escritura/lectura es atendida) y

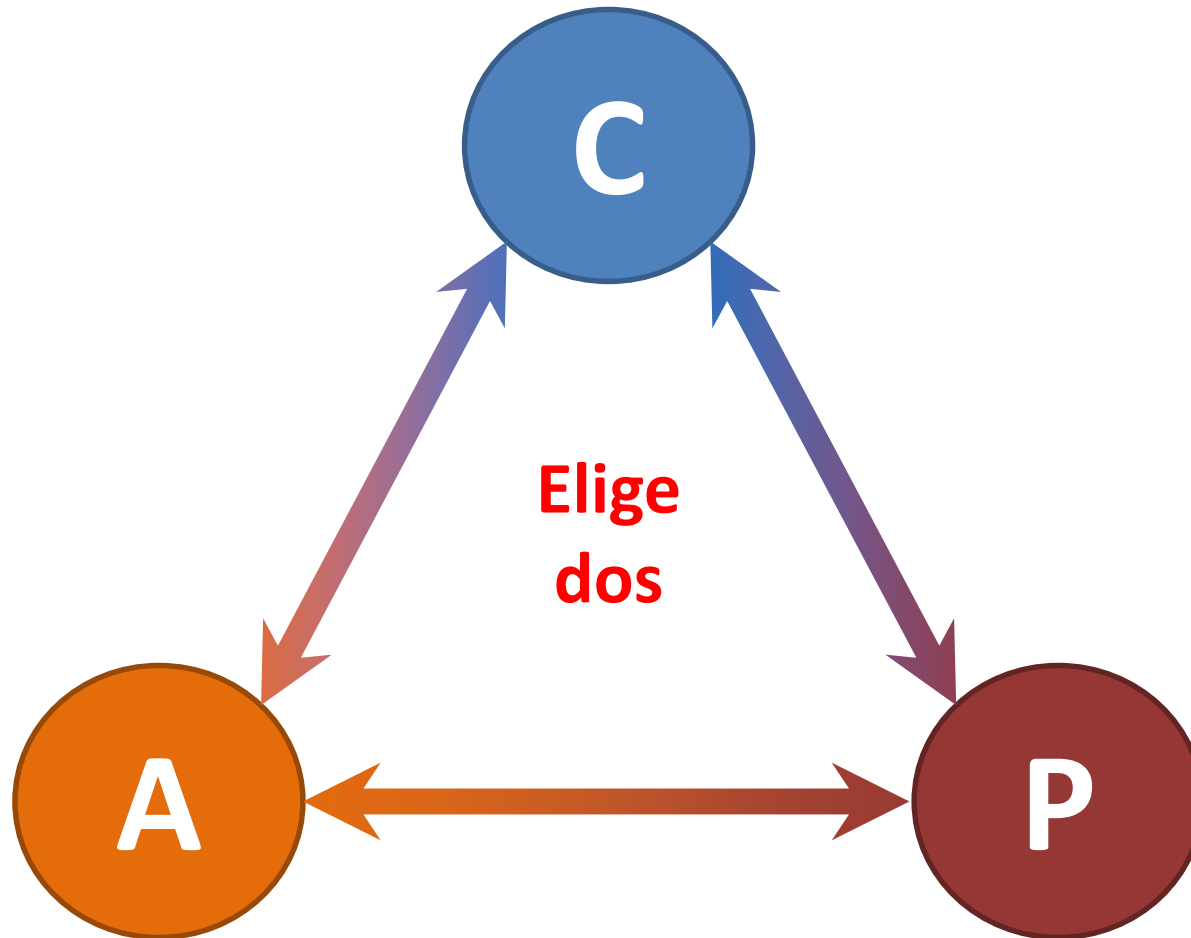
**tolerancia a Particiones** (el sistema funciona incluso si hay mensajes perdidos)  
al mismo tiempo!



# La "demostración" CAP



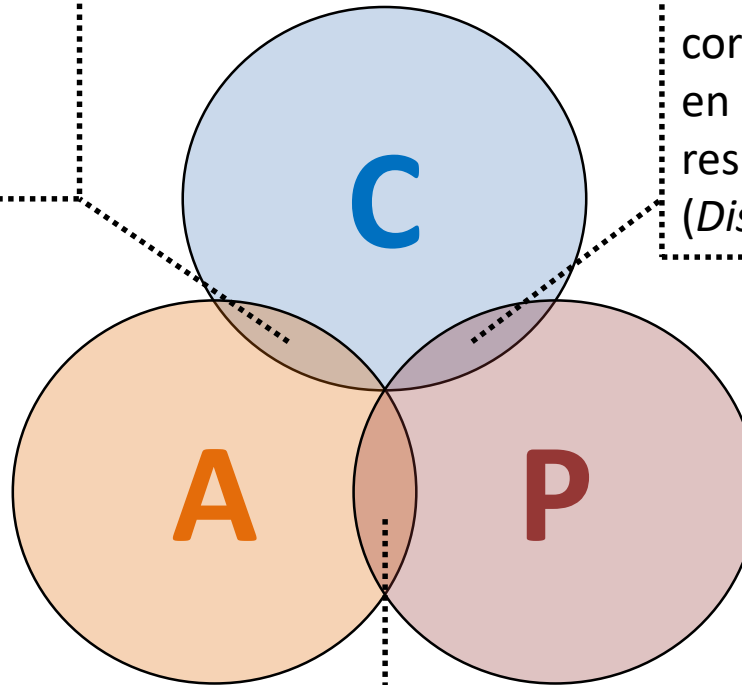
# El triángulo CAP



# Sistemas CAP

**CA**: Garantiza respuestas correctas sólo mientras la conexión funcione bien  
(*Centralizado / Tradicional*)

**CP**: Garantiza respuestas correctas incluso si hay fallas en la conexión, pero la respuesta puede fracasar  
(*Disponibilidad débil*)



(No hay intersección)

**AP**: Siempre provee "la mejor" respuesta que puede incluso en presencia de fallas en la conexión  
(*Coherencia eventual*)

# Sistema CA

892

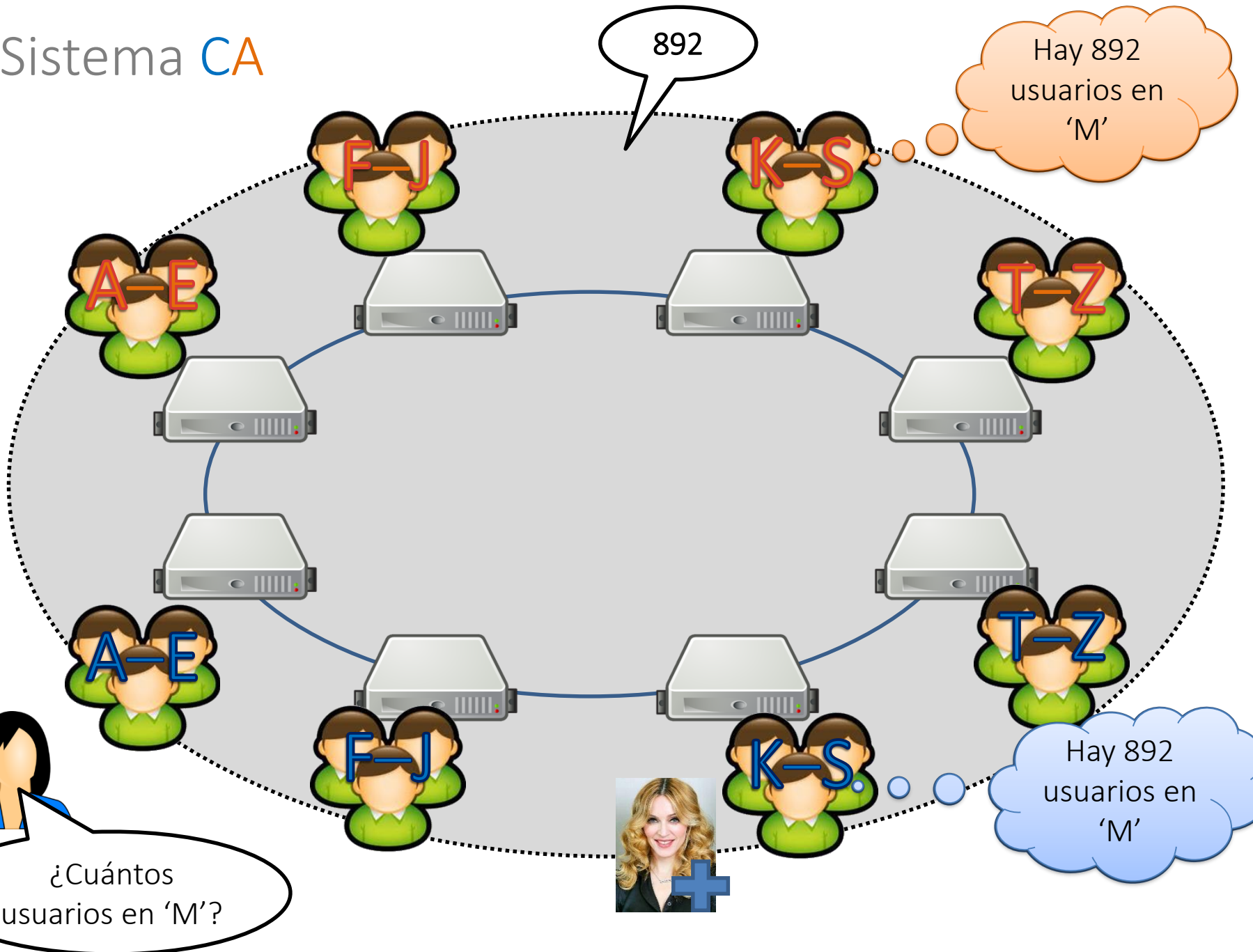
Hay 892  
usuarios en  
'M'



¿Cuántos  
usuarios en 'M'?



Hay 892  
usuarios en  
'M'



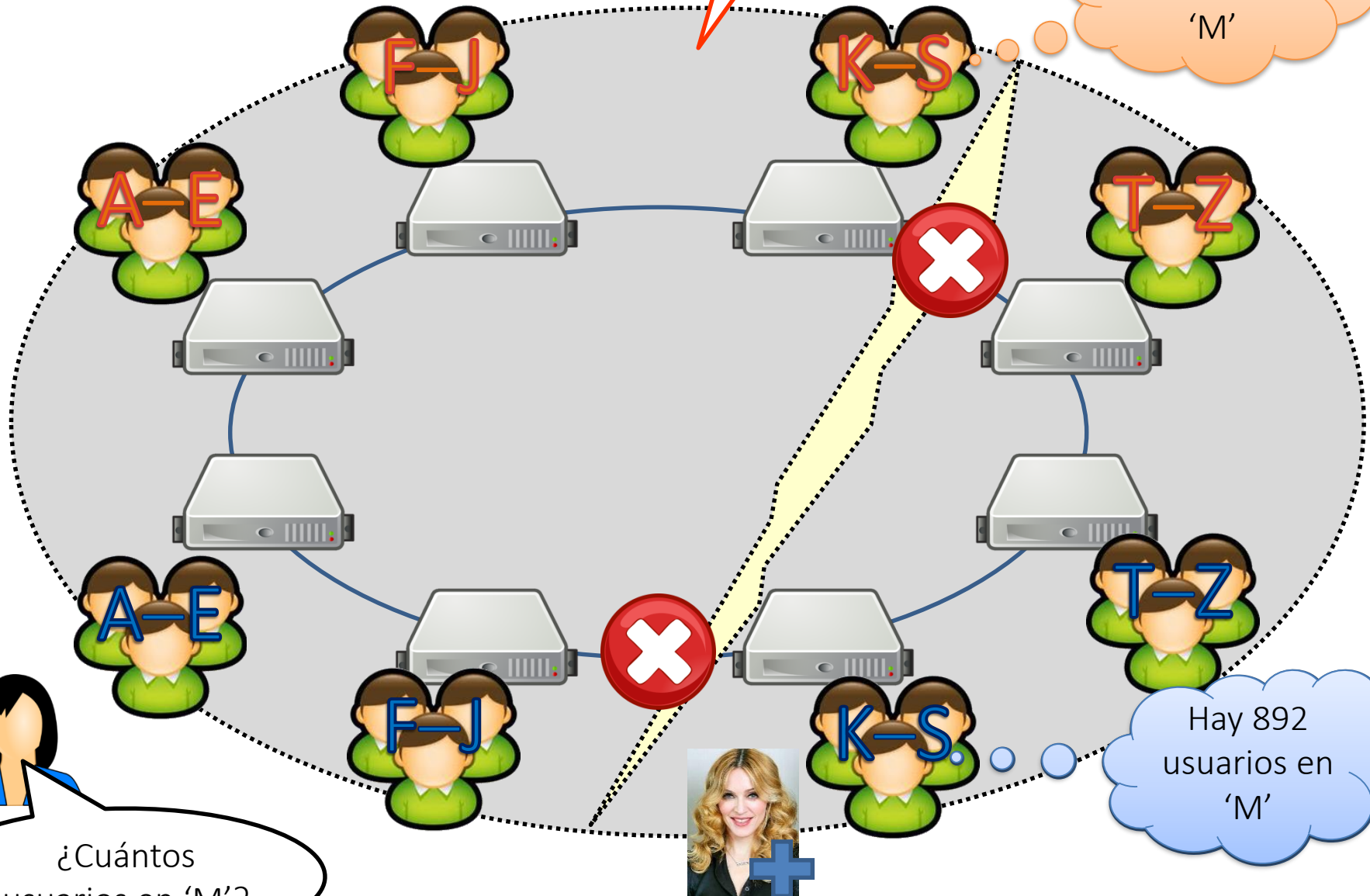
# Sistema CP

Error

Hay 891 usuarios en 'M'

Hay 892 usuarios en 'M'

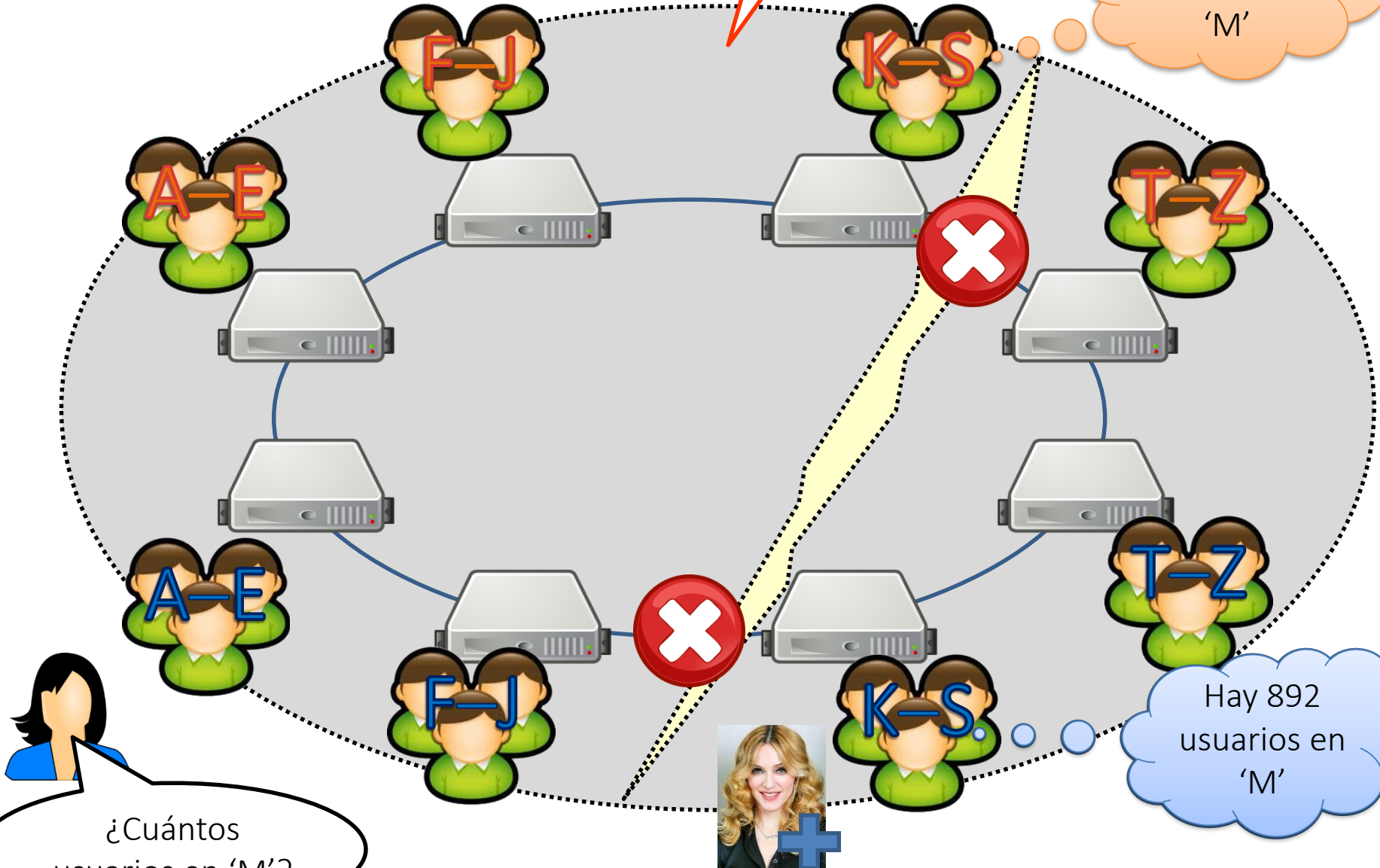
¿Cuántos usuarios en 'M'?



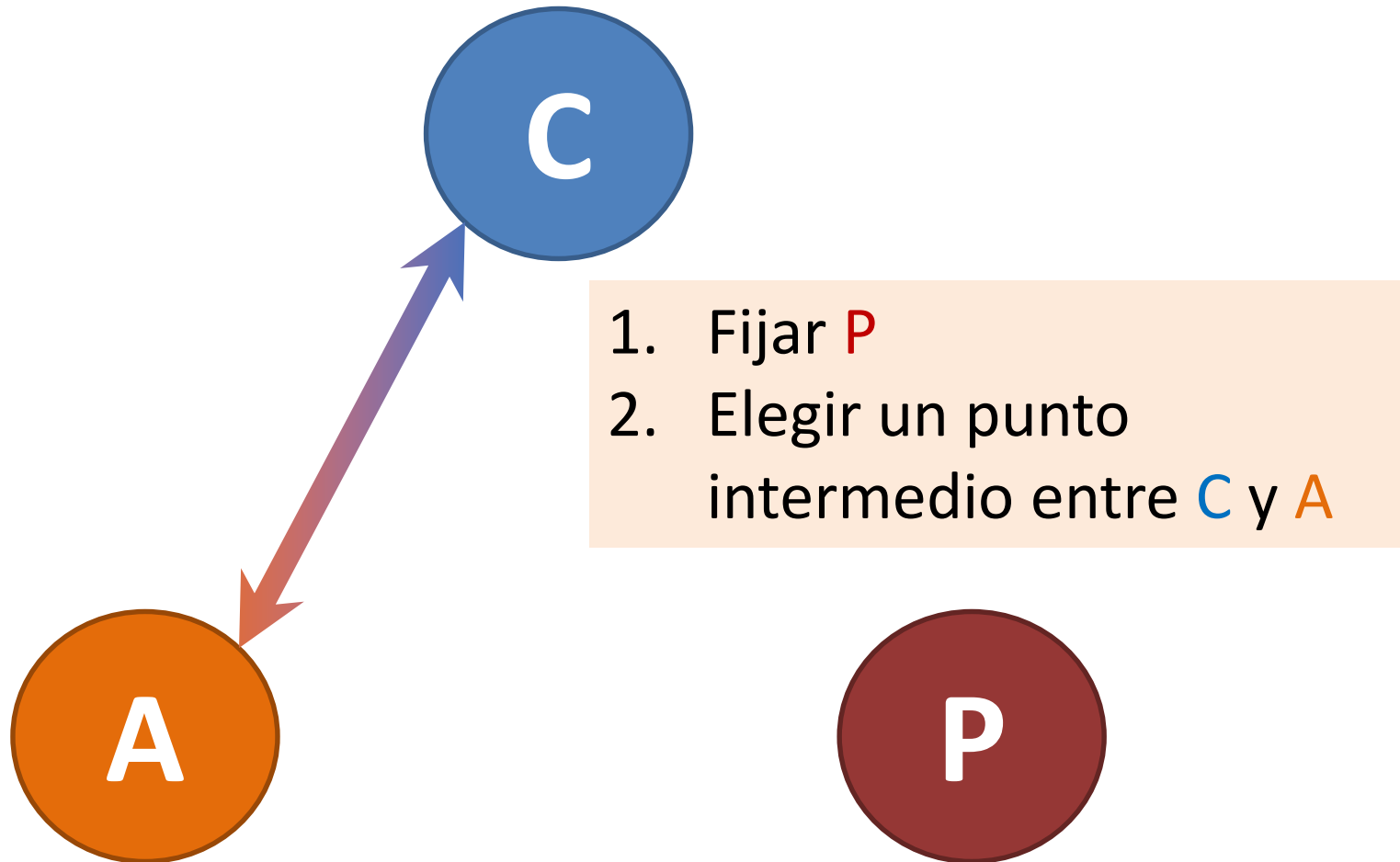
# Sistema AP

891

Hay 891  
usuarios en  
'M'



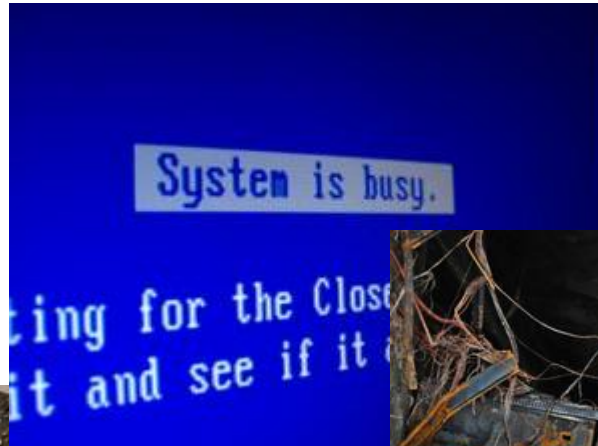
# CAP en la práctica



TOLERANCIA A PARTICIONES

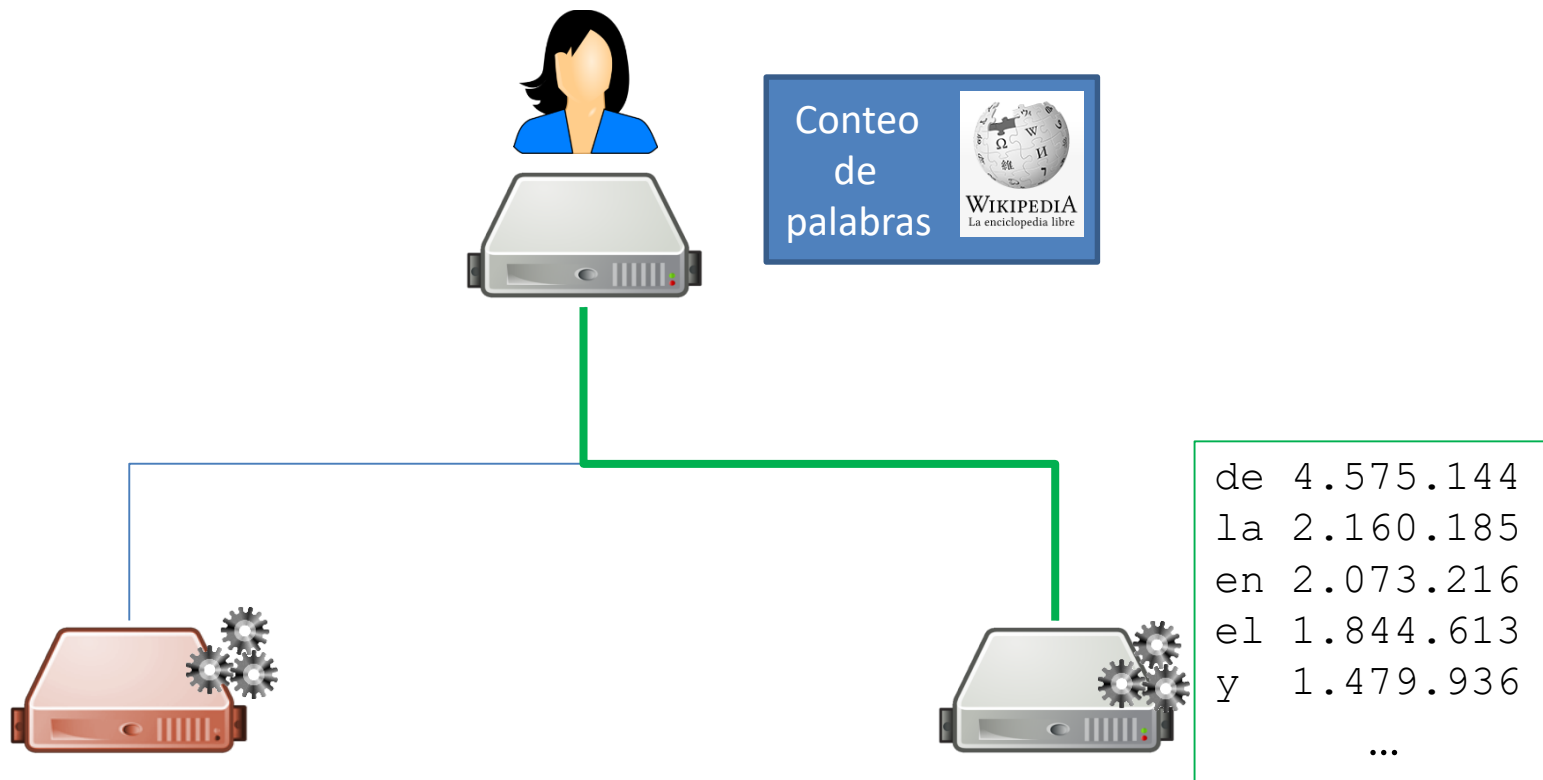


# Fallas



# Falla "Fail-Stop"

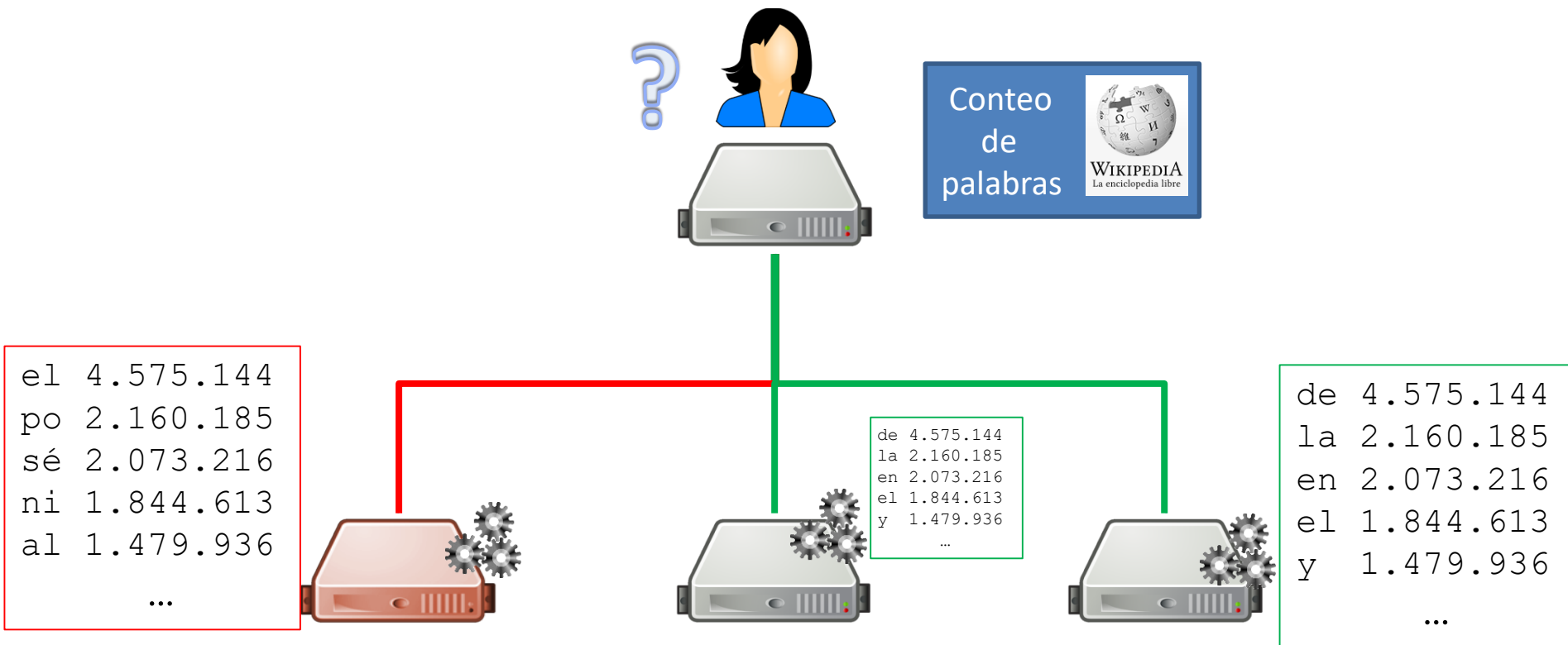
- Una máquina no responde (a tiempo)
  - a menudo, por falla del hardware o por sobrecarga
  - se necesita al menos  $f + 1$  máquinas replicadas
    - $f$  = número de fallas "fail-stop"



# Falla Bizantina

- Una máquina responde de forma incorrecta

¿Cuántas máquinas operativas necesitamos para corregir una falla bizantina?



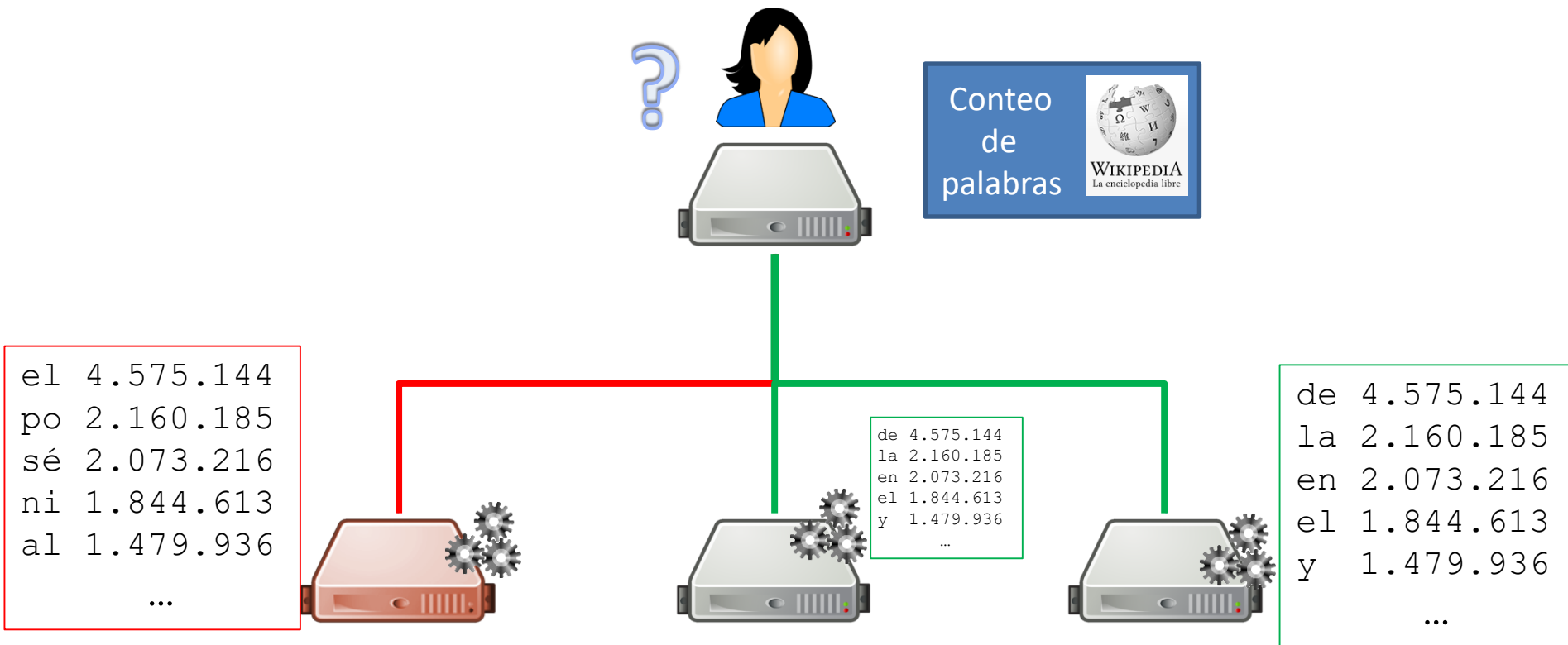
# Falla Bizantina

- Una máquina responde de forma incorrecta

¿Cuántas máquinas operativas necesitamos para corregir una falla bizantina? (?)

Se necesitan  $2f+1$  máquinas replicadas

- $f$  = número de fallas (posiblemente) bizantinas



¿Consenso distribuido?

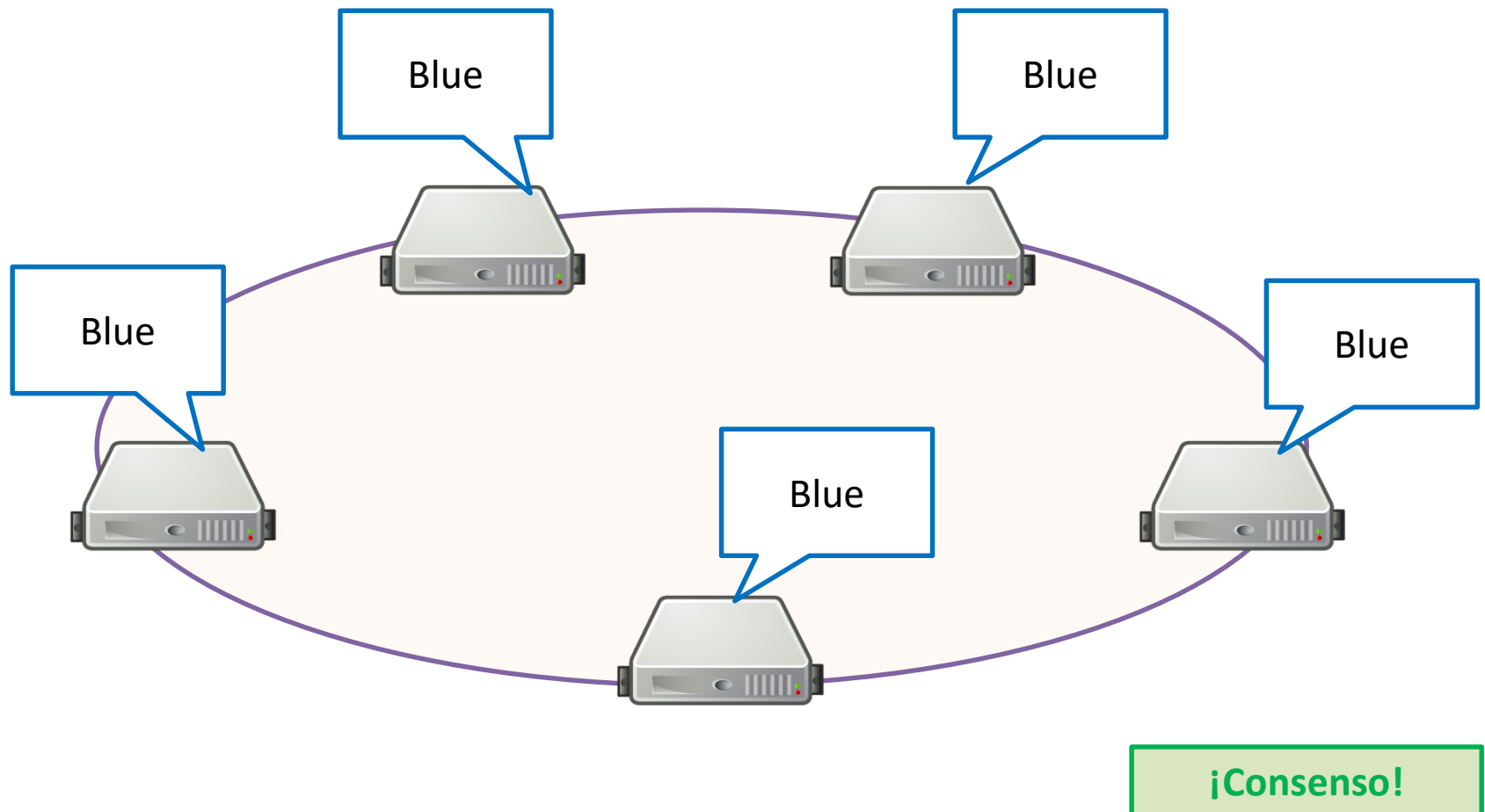


¿Color del vestido?



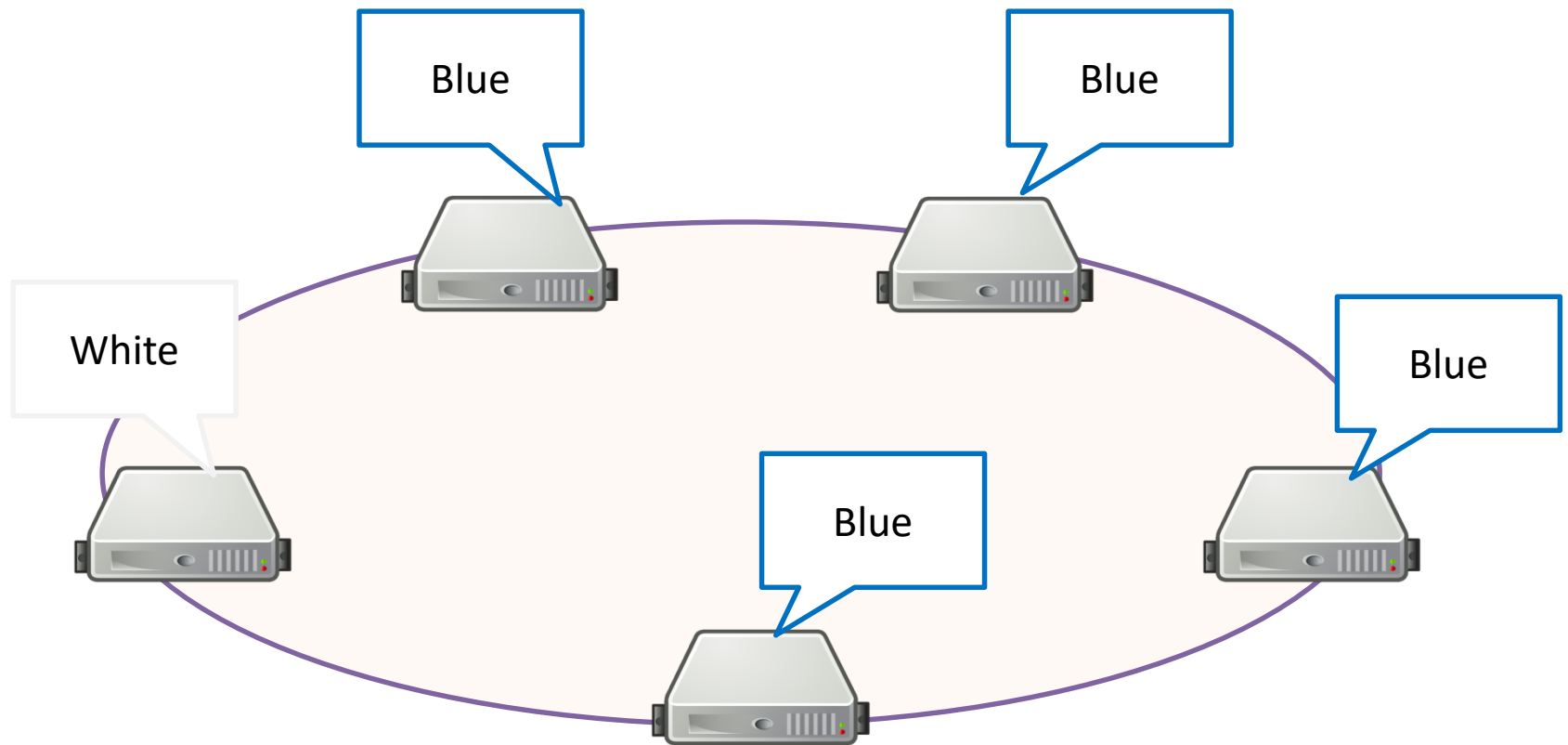
# Consenso distribuido

Consenso fuerte: Todos los nodos deben concordar



# Consenso distribuido

Consenso fuerte: Todos los nodos deben concordar

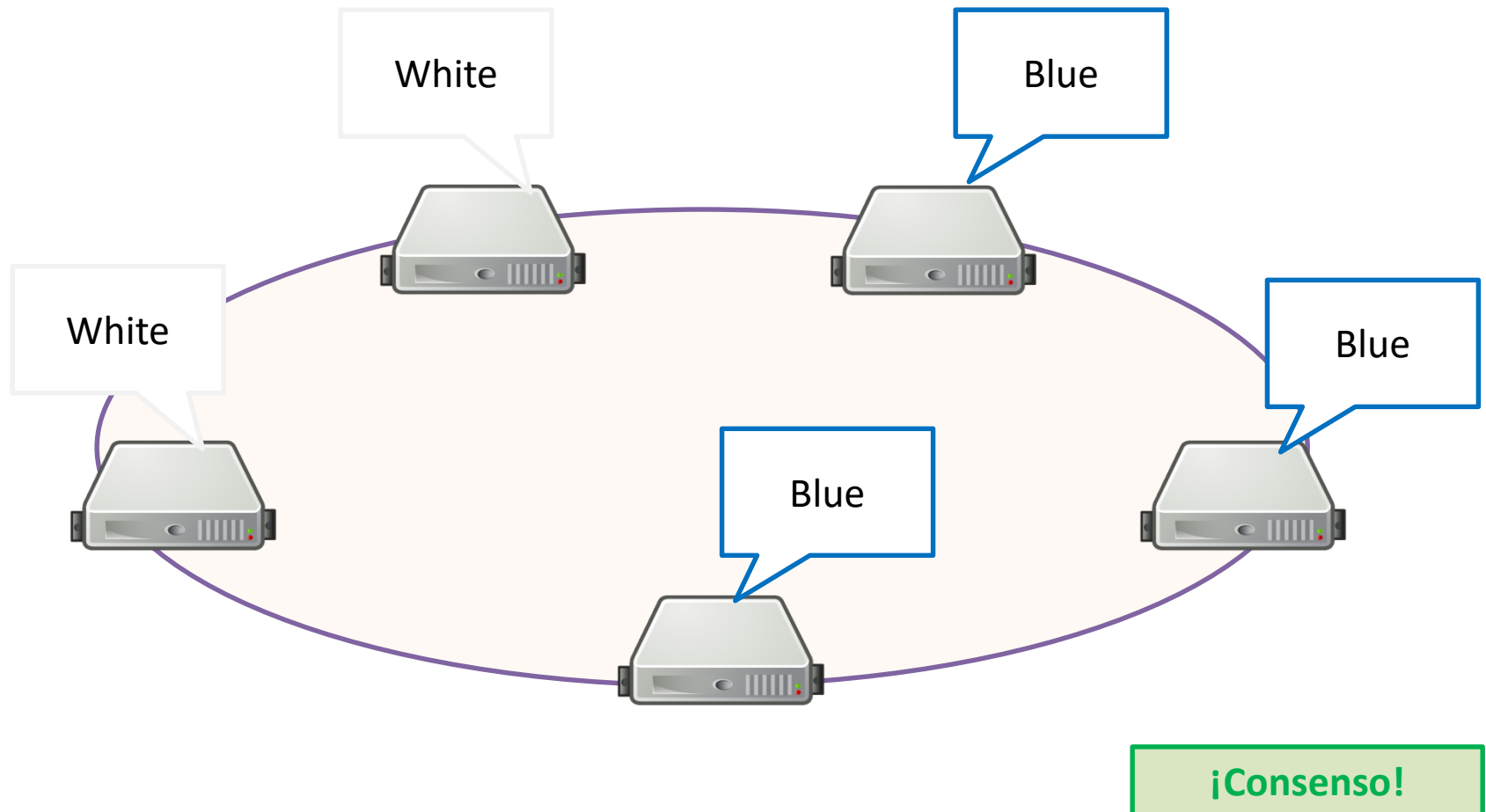


**¡Sin consenso!**



# Consenso distribuido

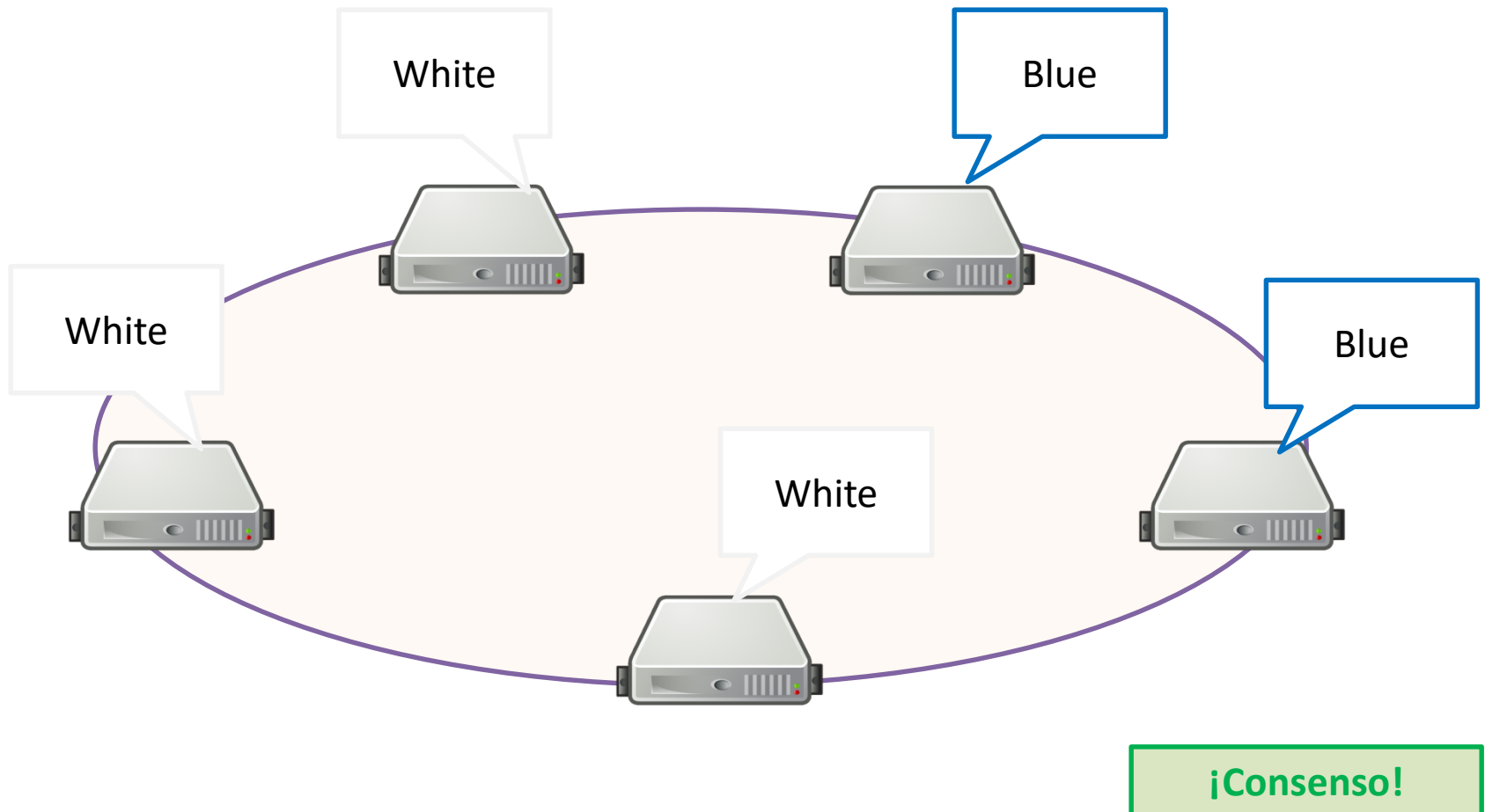
Consenso mayoritario: Debe concordar una mayoría





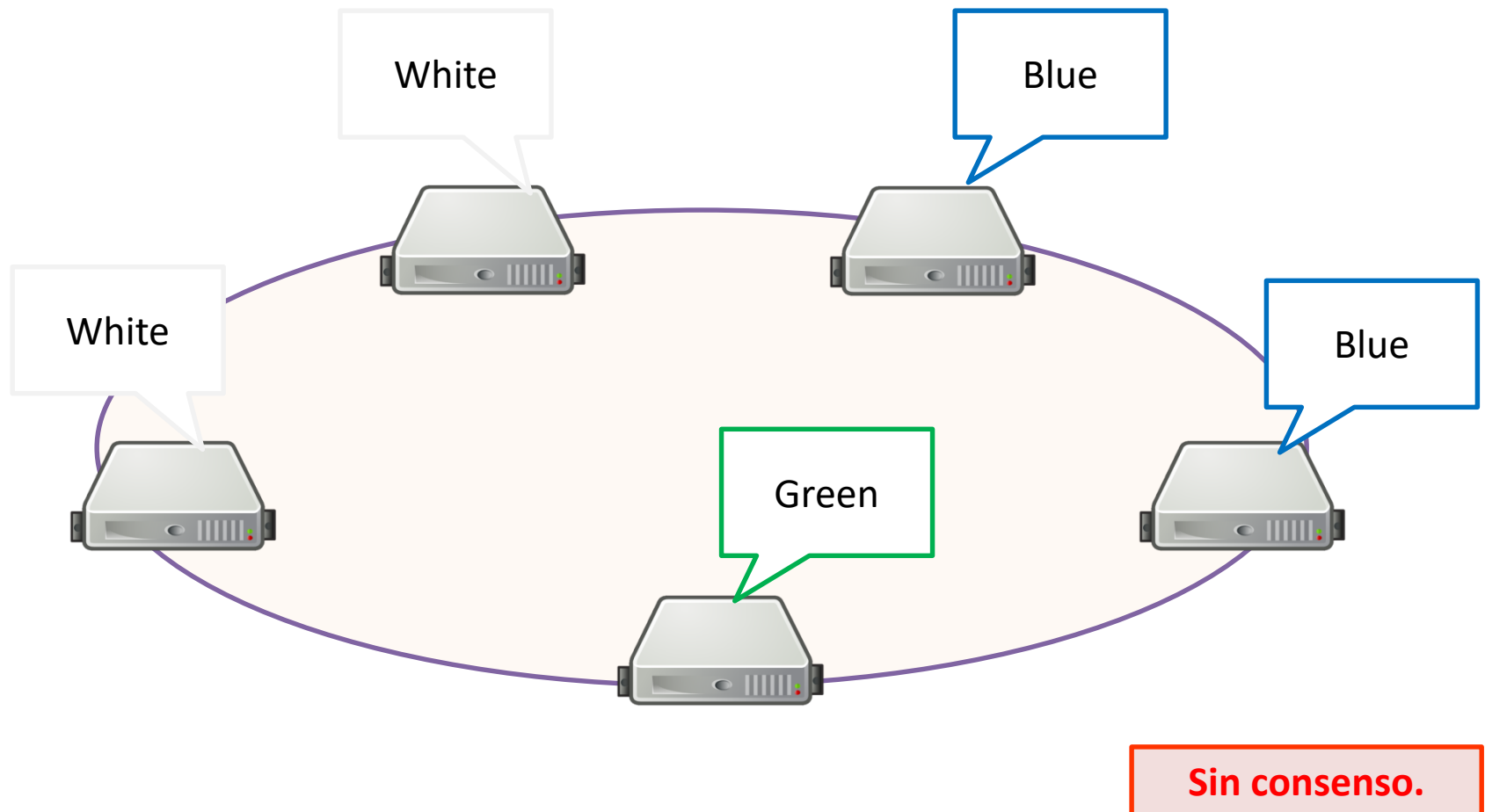
# Consenso distribuido

Consenso mayoritario: Debe concordar una mayoría



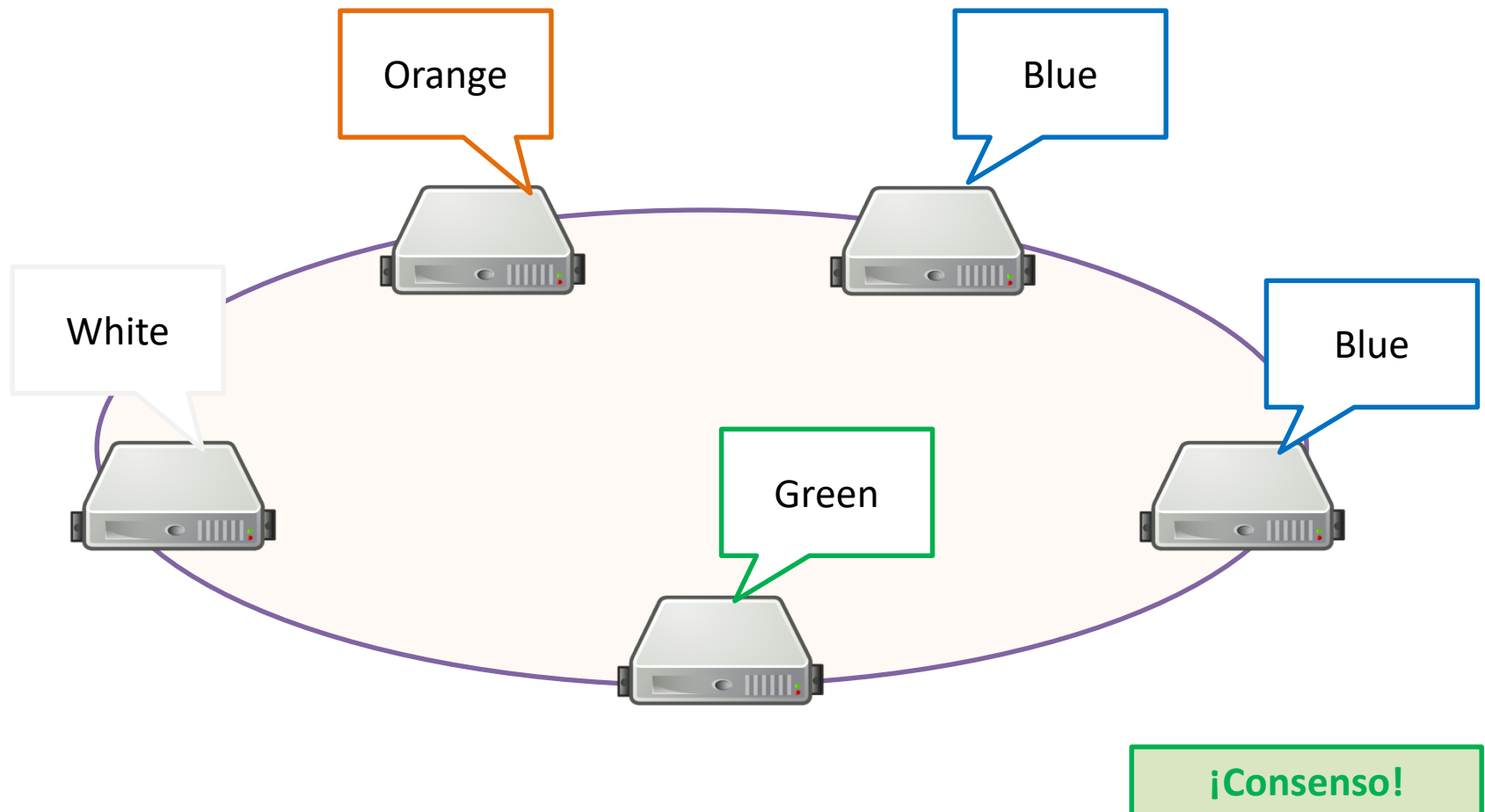
# Consenso distribuido

Consenso mayoritario: Debe concordar una mayoría



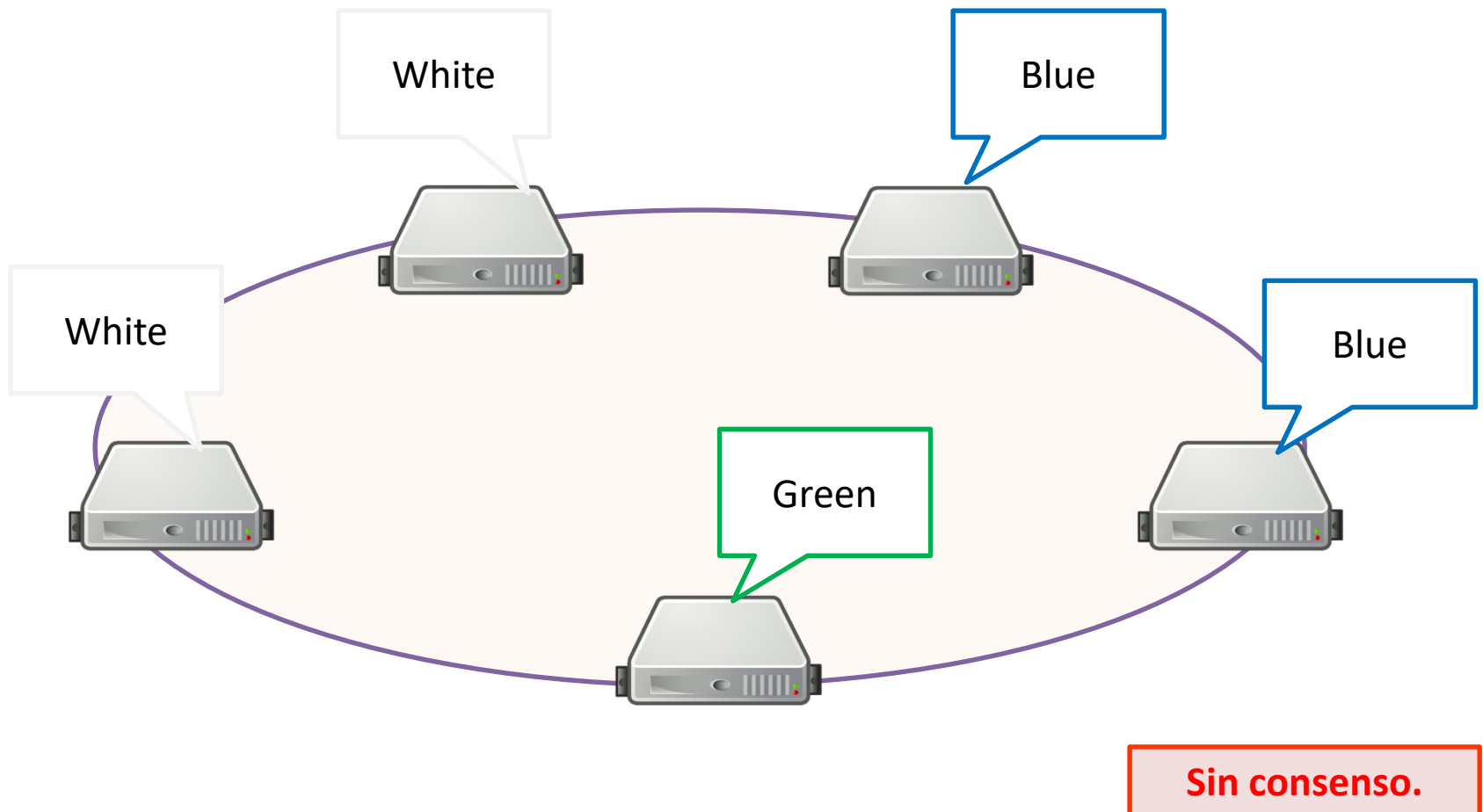
# Consenso distribuido

Consenso plural: Debe concordar una pluralidad



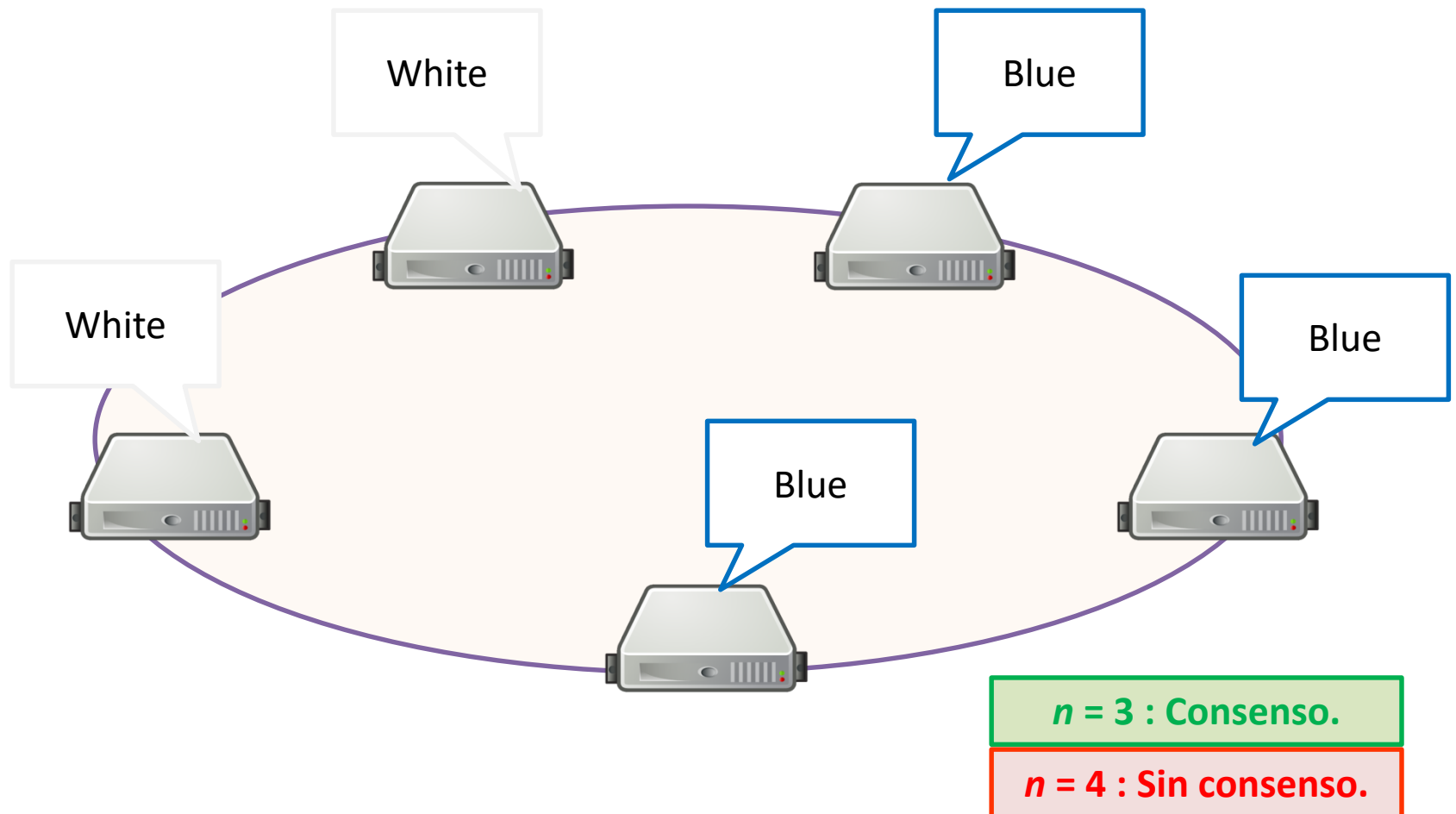
# Consenso distribuido

Consenso plural: Debe concordar una pluralidad



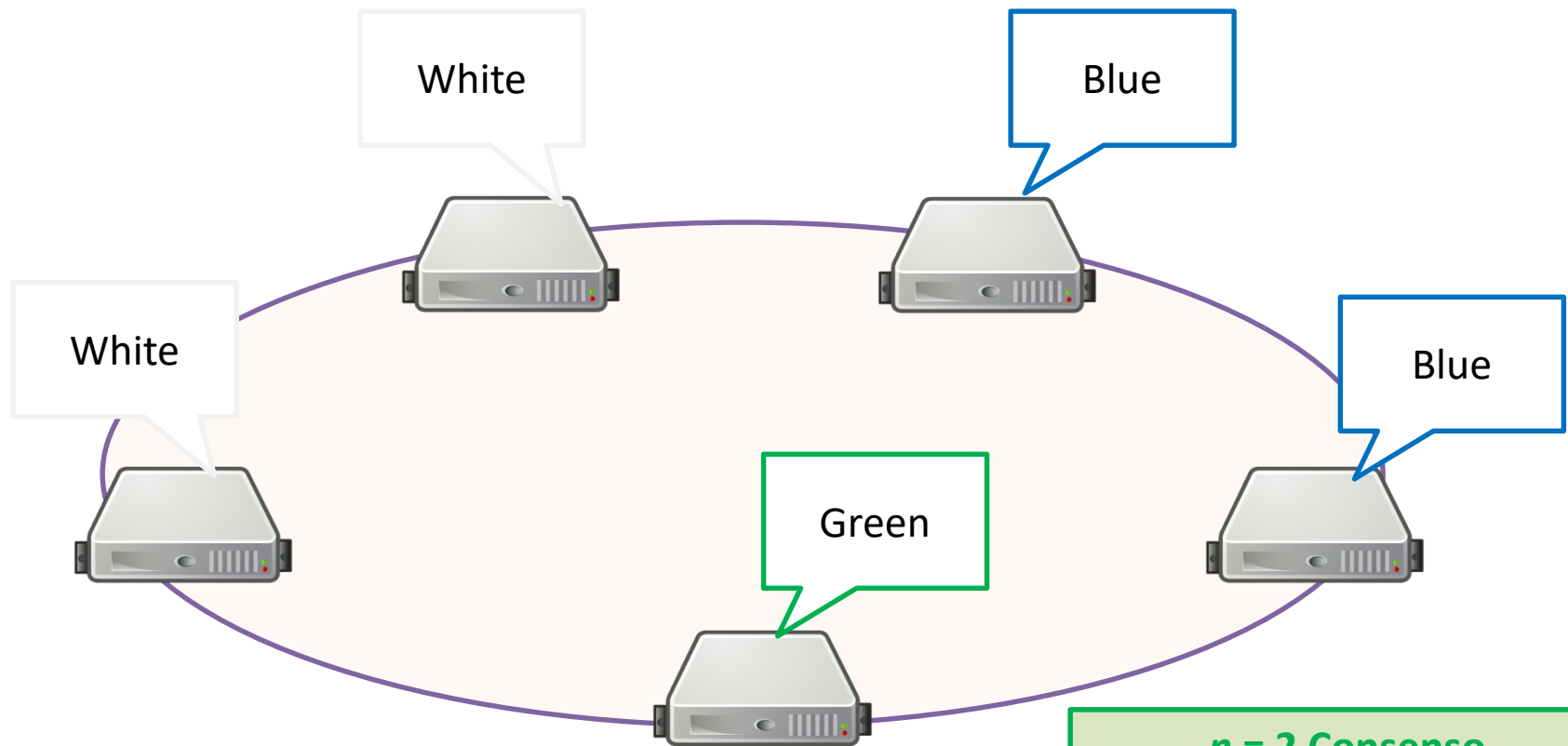
# Consenso distribuido

Consenso de quorum:  $n$  nodos deben concordar



# Consenso distribuido

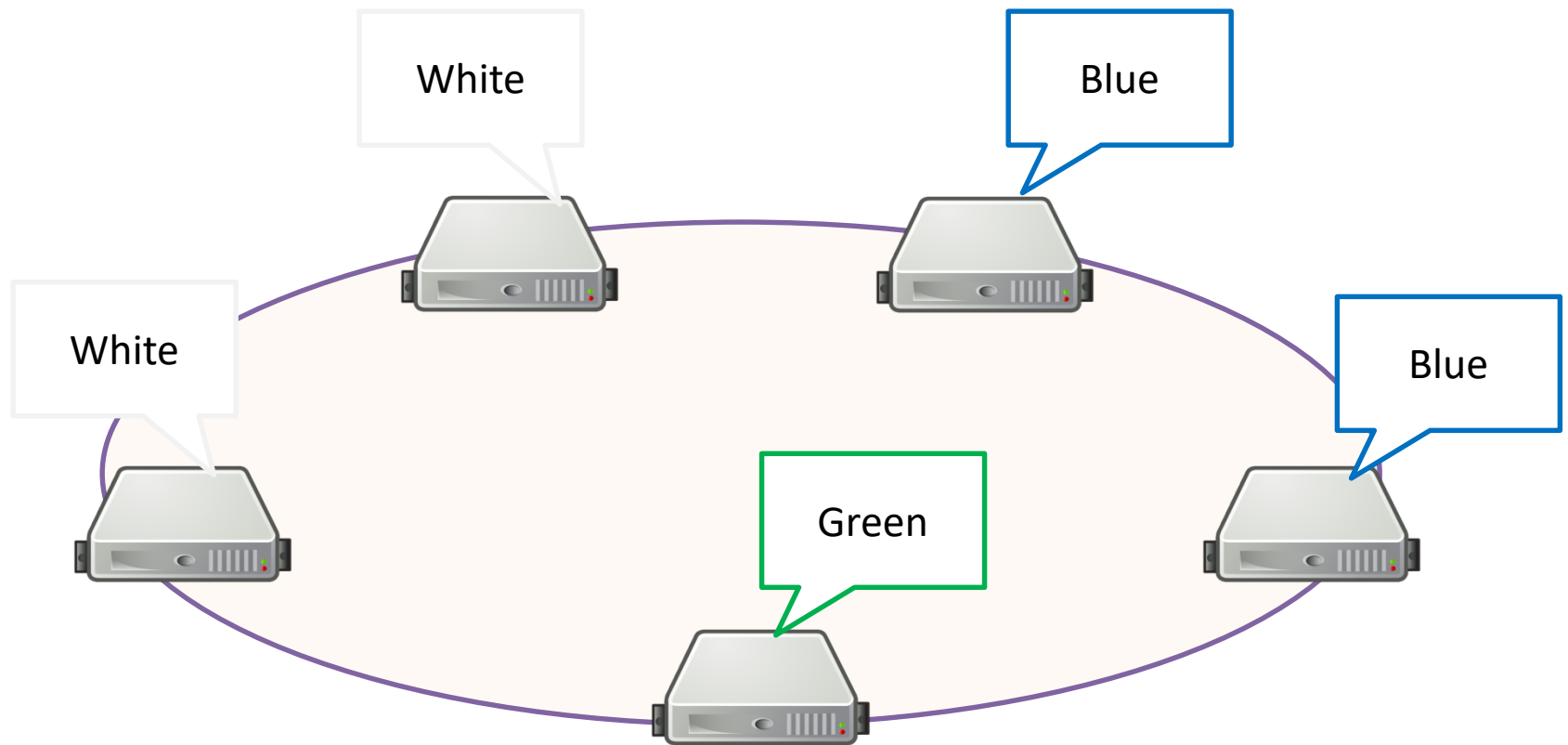
Consenso de quorum:  $n$  nodos deben concordar



**$n = 2$  Consenso.**  
(Primeras 2 máquinas consultadas; ¡no es único!)

# Consenso distribuido

Consenso de quorum:  $n$  nodos deben concordar

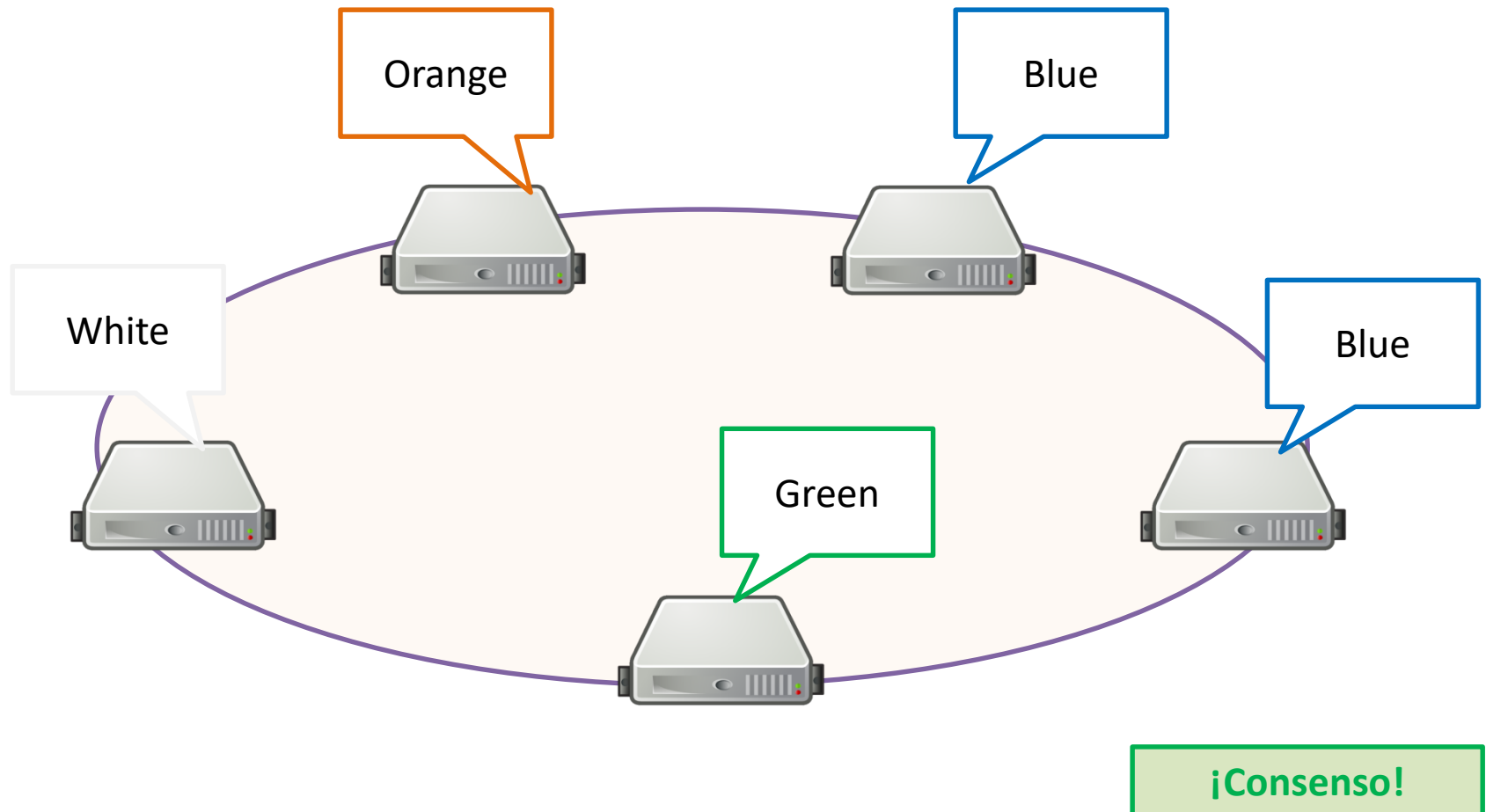


¿Valor de  $n$  necesario para un consenso único con  $N$  nodos? (?)

$$n > N/2$$

# Consenso distribuido

Consenso deshabilitado: Toma el primer resultado





# Consenso distribuido

¿CP vs. AP?



Consenso fuerte: Todos los nodos deben concordar

CP

Consenso mayoritario: Deben concordar una mayoría

Consenso plural: Deben concordar una pluralidad

Consenso de quorum: Deben concordar  $n$  nodos

Consenso deshabilitado: Toma el primer resultado

AP

# Consenso distribuido

¿La escalabilidad?



Consenso fuerte: Todos los nodos deben concordar

**Más mensajes**

Consenso mayoritario: Deben concordar una mayoría

Consenso plural: Deben concordar una pluralidad

Consenso de quorum: Deben concordar  $n$  nodos

Consenso deshabilitado: Toma el primer resultado

**Menos mensajes**

# Consenso distribuido

Consenso fuerte: Todos los nodos deben concordar

Consenso mayoritario: Deben concordar una mayoría

**La mejor elección depende de la aplicación:**

**Muchos sistemas NoSQL permiten elegir  
el nivel de consenso/replicación**

Consenso plural: Deben concordar una pluralidad

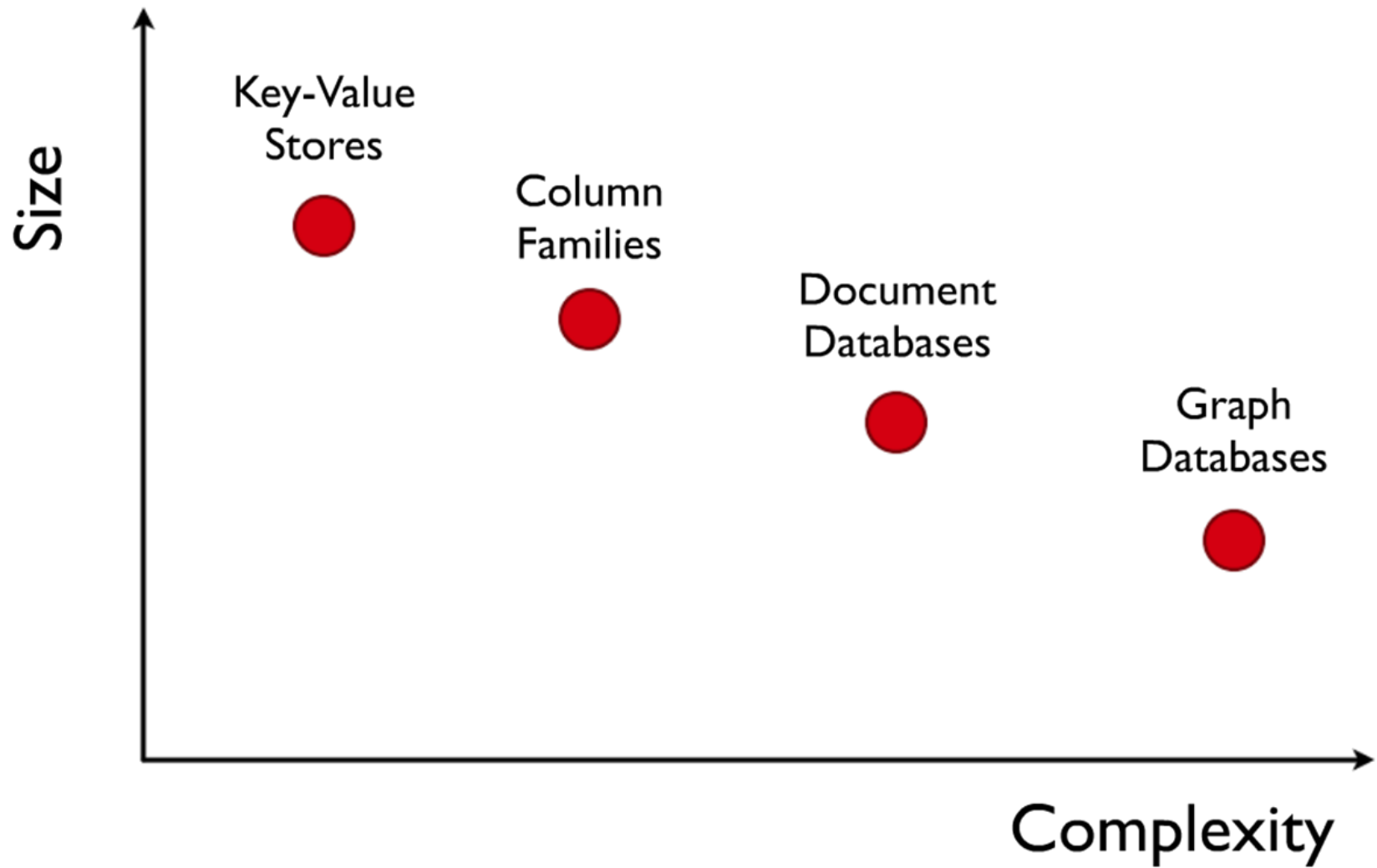
Consenso de quorum: Deben concordar  $n$  nodos

Consenso deshabilitado: Toma el primer resultado

NoSQL:

SISTEMAS DE LLAVE—VALOR ("KEY—VALUE")

# NoSQL



# Modelo de Llave–Valor

Es sólo un mapa 😊

- `put(key, value)`
- `get(key)`
- `delete(key)`

| Key                 | Value            |
|---------------------|------------------|
| Afghanistan         | Kabul            |
| Albania             | Tirana           |
| Algeria             | Algiers          |
| Andorra la vella    | Andorra la vella |
| Angola              | Luanda           |
| Antigua and Barbuda | St. John's       |
| ...                 | ...              |

Pero se puede hacer mucho con un mapa

| Key                 | Value                                                 |
|---------------------|-------------------------------------------------------|
| country:Afghanistan | capital@city:Kabul,continent:Asia,pop:31108077#2011   |
| country:Albania     | capital@city:Tirana,continent:Europe,pop:3011405#2013 |
| ...                 | ...                                                   |
| city:Kabul          | country:Afghanistan,pop:3476000#2013                  |
| city:Tirana         | country:Albania,pop:3011405#2013                      |
| ...                 | ...                                                   |
| user:10239          | basedIn@city:Tirana,post:{103,10430,201}              |
| ...                 | ...                                                   |

Para los interesados ...



## **Dynamo: Amazon's Highly Available Key-value Store**

Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati,  
Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Voss  
and Werner Vogels

Amazon.com

### **ABSTRACT**

Reliability at massive scale is one of the biggest challenges we face at Amazon.com, one of the largest e-commerce operations in the world; even the slightest outage has significant financial consequences and impacts customer trust. The Amazon.com platform, which provides services for many web sites worldwide, is implemented on top of an infrastructure of tens of thousands of servers and network components located in many datacenters

One of the lessons our organization has learned from operating Amazon's platform is that the reliability and scalability of a system is dependent on how its application state is managed. Amazon uses a highly decentralized, loosely coupled, service oriented architecture consisting of hundreds of services. In this environment there is a particular need for storage technologies that are always available. For example, customers should be able to view and add items to their shopping cart even if disks are failing, network routes are flapping, or data centers are being



OTROS GUARDADOS LLAVE-VALOR

# Otros sistemas de guardado llave-valor

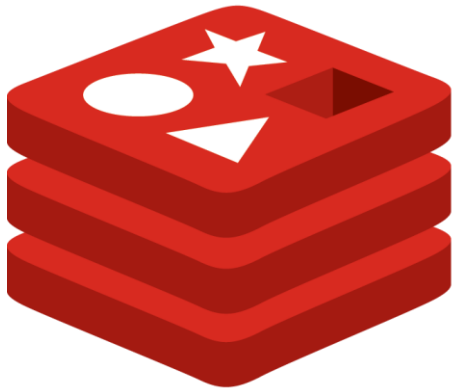


at&t



**Aol.**

# Otros sistemas de guardado llave-valor



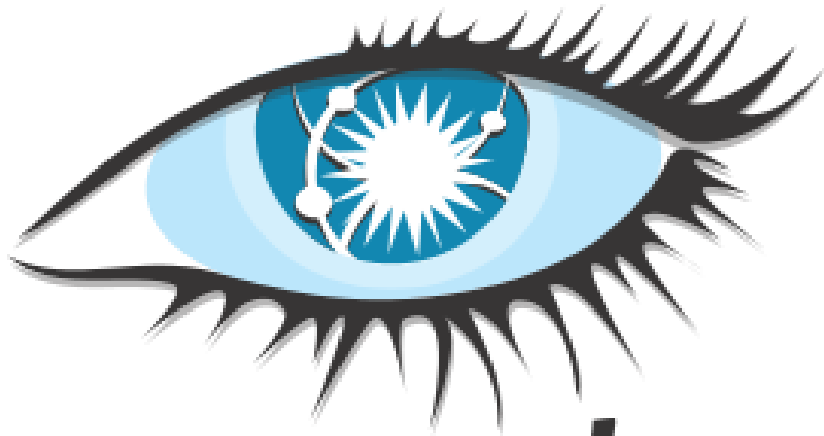
redis



StackExchange



# Otros sistemas de guardado llave-valor



***cassandra***



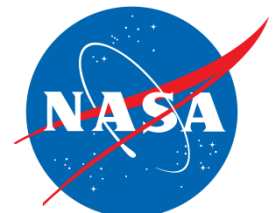
**Instagram**  
Fast beautiful photo sharing

**accenture**  
High performance. Delivered.

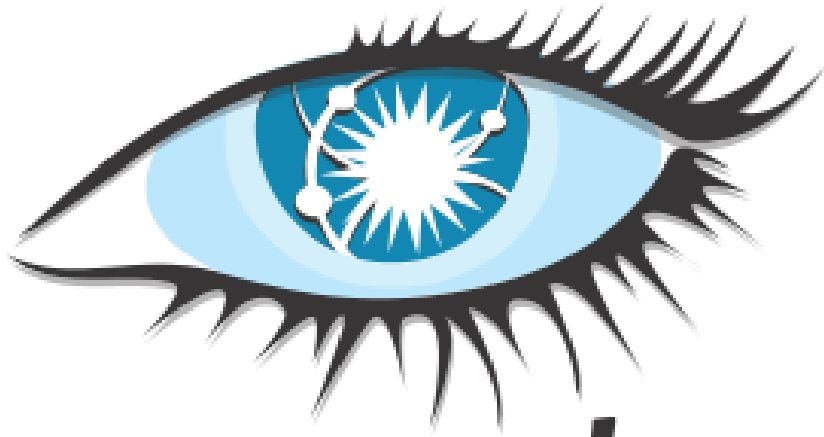
**Answers.com**



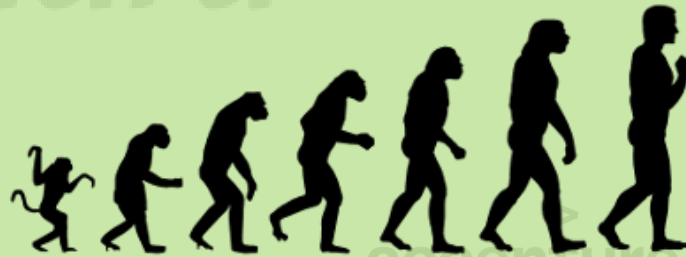
**Disney**



# Otros sistemas de guardado llave-valor



cassandra



Instagram  
Fast beautiful photo sharing

accenture  
High performance. Delivered.

Answers.com

Ahora uso un modelo tabular



Disney



| Rank     |          |          | DBMS                         | Database Model             | Score    |          |          |
|----------|----------|----------|------------------------------|----------------------------|----------|----------|----------|
| Jul 2025 | Jun 2025 | Jul 2024 |                              |                            | Jul 2025 | Jun 2025 | Jul 2024 |
| 1.       | 1.       | 1.       | Oracle                       | Relational, Multi-model ⓘ  | 1217.05  | -13.33   | -23.31   |
| 2.       | 2.       | 2.       | MySQL                        | Relational, Multi-model ⓘ  | 940.73   | -12.85   | -98.73   |
| 3.       | 3.       | 3.       | Microsoft SQL Server         | Relational, Multi-model ⓘ  | 771.14   | -5.61    | -36.51   |
| 4.       | 4.       | 4.       | PostgreSQL                   | Relational, Multi-model ⓘ  | 680.89   | +0.23    | +41.98   |
| 5.       | 5.       | 5.       | MongoDB 📦                    | Document, Multi-model ⓘ    | 403.83   | +0.99    | -25.99   |
| 6.       | 6.       | 📈 7.     | Snowflake                    | Relational                 | 176.17   | +1.68    | +39.64   |
| 7.       | 7.       | 📉 6.     | Redis                        | Key-value, Multi-model ⓘ   | 149.72   | -2.01    | -7.05    |
| 8.       | 8.       | 📈 9.     | IBM Db2                      | Relational, Multi-model ⓘ  | 127.51   | +2.38    | +3.11    |
| 9.       | 9.       | 📉 8.     | Elasticsearch                | Multi-model ⓘ              | 118.83   | -2.45    | -12.00   |
| 10.      | 10.      | 10.      | SQLite                       | Relational                 | 115.44   | -1.60    | +5.49    |
| 11.      | 11.      | 📈 12.    | Apache Cassandra             | Wide column, Multi-model ⓘ | 108.76   | +0.49    | +9.63    |
| 12.      | 12.      | 📈 15.    | Databricks                   | Multi-model ⓘ              | 108.04   | +3.36    | +24.74   |
| 13.      | 13.      | 📈 14.    | MariaDB 📦                    | Relational, Multi-model ⓘ  | 95.44    | +0.91    | +4.86    |
| 14.      | 14.      | 📉 11.    | Microsoft Access             | Relational                 | 90.47    | +2.19    | -10.17   |
| 15.      | 15.      | 📈 17.    | Amazon DynamoDB              | Multi-model ⓘ              | 84.15    | +0.81    | +13.20   |
| 16.      | 📈 17.    | 16.      | Microsoft Azure SQL Database | Relational, Multi-model ⓘ  | 76.51    | +1.20    | -0.24    |
| 17.      | 📉 16.    | 📈 19.    | Apache Hive                  | Relational                 | 75.74    | -0.94    | +18.45   |
| 18.      | 18.      | 📉 13.    | Splunk                       | Search engine              | 69.52    | -0.10    | -23.40   |
| 19.      | 19.      | 📉 18.    | Google BigQuery              | Relational                 | 64.19    | -0.35    | +6.38    |
| 20.      | 20.      | 📈 21.    | Neo4j                        | Graph                      | 54.64    | +3.30    | +8.88    |

MODELO TABULAR / FAMILIA DE COLUMNAS

Llave–Valor = un mapa distribuido

| Countries   |                                                  |
|-------------|--------------------------------------------------|
| Primary Key | Value                                            |
| Afghanistan | capital:Kabul,continent:Asia,pop:31108077#2011   |
| Albania     | capital:Tirana,continent:Europe,pop:3011405#2013 |
| ...         | ...                                              |

Tabular = un mapa multi-dimensional

| Countries   |         |           |           |          |
|-------------|---------|-----------|-----------|----------|
| Primary Key | capital | continent | pop-value | pop-year |
| Afghanistan | Kabul   | Asia      | 31108077  | 2011     |
| Albania     | Tirana  | Europe    | 3011405   | 2013     |
| ...         | ...     | ...       | ...       | ...      |



Para los interesados ...



## **Bigtable: A Distributed Storage System for Structured Data**

Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach  
Mike Burrows, Tushar Chandra, Andrew Fikes, Robert E. Gruber

{fay,jeff,sanjay,wilsonh,kerr,m3b,tushar,fikes,gruber}@google.com

*Google, Inc.*

### **Abstract**

Bigtable is a distributed storage system for managing structured data that is designed to scale to a very large size: petabytes of data across thousands of commodity servers. Many projects at Google store data in Bigtable, including web indexing, Google Earth, and Google Finance. These applications place very different demands on Bigtable, both in terms of data size (from URLs to web pages to satellite imagery) and latency requirements (from backend bulk processing to real-time data serving). Despite these varied demands, Bigtable has successfully provided a flexible, high-performance solution for all of these Google products. In this paper we describe the simple data model provided by Bigtable, which gives clients dynamic control over data layout and format, and we describe the design and implementation of Bigtable.

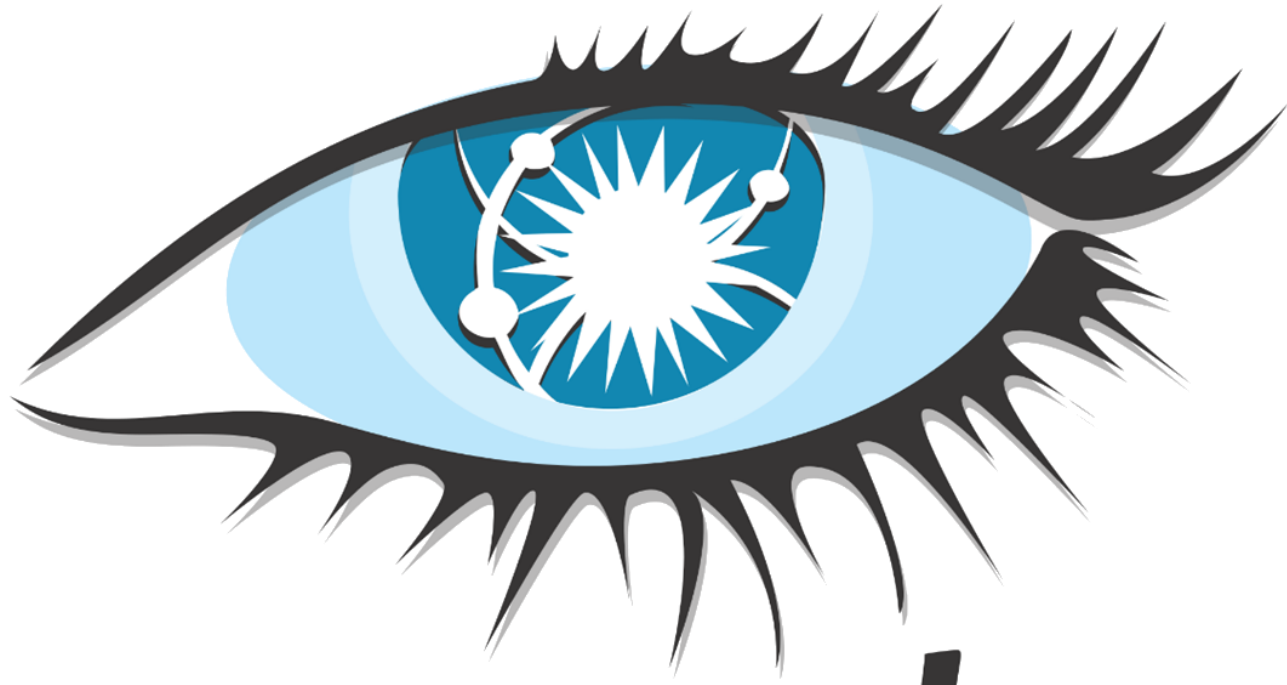
achieved scalability and high performance, but Bigtable provides a different interface than such systems. Bigtable does not support a full relational data model; instead, it provides clients with a simple data model that supports dynamic control over data layout and format, and allows clients to reason about the locality properties of the data represented in the underlying storage. Data is indexed using row and column names that can be arbitrary strings. Bigtable also treats data as uninterpreted strings, although clients often serialize various forms of structured and semi-structured data into these strings. Clients can control the locality of their data through careful choices in their schemas. Finally, Bigtable schema parameters let clients dynamically control whether to serve data out of memory or from disk.

Section 2 describes the data model in more detail, and

Tabular: Apache HBase



Tabular: Cassandra



***cassandra***

CASSANDRA

# Almacena Tabular

## Ventas

| producto   | tienda      | cliente | fecha                | valor |
|------------|-------------|---------|----------------------|-------|
| 1L_Leche   | Santiago    | 412     | 2020-03-31T08:47:57Z | 900   |
| La_Tercera | Providencia | 413     | 2020-03-31T08:47:59Z | 2000  |
| Nescafe    | Providencia | 413     | 2020-03-31T08:47:59Z | 3500  |
| Nescafe    | Providencia | 413     | 2020-03-31T08:48:00Z | 3500  |
| Comfort    | Valparaíso  | 414     | 2020-03-31T08:48:04Z | 2300  |
| Nescafe    | Providencia | 415     | 2020-03-31T08:48:04Z | 3500  |
| Comfort    | Valparaíso  | 416     | 2020-03-31T08:48:07Z | 2300  |
| 1L_Leche   | Valparaíso  | 416     | 2020-03-31T08:48:07Z | 800   |
| Nescafe    | Santiago    | 412     | 2020-03-31T08:48:08Z | 3700  |

# Cassandra Query Language (CQL)

```
SELECT [ JSON | DISTINCT ] ( select_clause  
| '*' )  
FROM table_name  
[ WHERE where_clause ]  
[ GROUP BY group_by_clause ]  
[ ORDER BY ordering_clause ]  
[ PER PARTITION LIMIT (integer |  
bind_marker) ]  
[ LIMIT (integer | bind_marker) ]  
[ ALLOW FILTERING ]
```

- **JSON:** Devolver datos en el formato de JSON
- **PER PARTITION LIMIT:** Limitar el número de filas por partición
- **ALLOW FILTERING:** Permitir consultas que puedan filtrar filas leídas

# Cassandra Query Language (CQL)

## Ventas

| producto   | tienda      | cliente | fecha                | valor |
|------------|-------------|---------|----------------------|-------|
| 1L_Leche   | Santiago    | 412     | 2020-03-31T08:47:57Z | 900   |
| La_Tercera | Providencia | 413     | 2020-03-31T08:47:59Z | 2000  |
| Nescafe    | Providencia | 413     | 2020-03-31T08:47:59Z | 3500  |
| Nescafe    | Providencia | 413     | 2020-03-31T08:48:00Z | 3500  |
| Confort    | Valparaíso  | 414     | 2020-03-31T08:48:04Z | 2300  |
| Nescafe    | Providencia | 415     | 2020-03-31T08:48:04Z | 3500  |
| Confort    | Valparaíso  | 416     | 2020-03-31T08:48:07Z | 2300  |
| 1L_Leche   | Valparaíso  | 416     | 2020-03-31T08:48:07Z | 800   |
| Nescafe    | Santiago    | 412     | 2020-03-31T08:48:08Z | 3700  |
| La_Tercera | Santiago    | 412     | 2020-03-31T08:48:08Z | 2000  |
| Comfort    | Santiago    | 412     | 2020-03-31T08:48:08Z | 2500  |
| ...        | ...         | ...     | ...                  | ...   |

```
SELECT * FROM Ventas WHERE producto = 'Confort' AND valor < 2500;
```

# Llave primaria

Ventas(**Partición** producto, tienda, **Ordenamiento** fecha, cliente, valor)



| <u>producto</u> | <u>tienda</u> | <u>fecha</u>         | <u>cliente</u> | valor |
|-----------------|---------------|----------------------|----------------|-------|
| 1L_Leche        | Santiago      | 2020-03-31T08:47:57Z | 412            | 900   |
| Confort         | Valparaíso    | 2020-03-31T08:48:04Z | 414            | 2300  |
| Confort         | Valparaíso    | 2020-03-31T08:48:07Z | 416            | 2300  |



| <u>producto</u> | <u>tienda</u> | <u>fecha</u>         | <u>cliente</u> | valor |
|-----------------|---------------|----------------------|----------------|-------|
| La_Tercera      | Providencia   | 2020-03-31T08:47:59Z | 413            | 2000  |
| Confort         | Santiago      | 2020-03-31T08:48:08Z | 412            | 2500  |



| <u>producto</u> | <u>tienda</u> | <u>fecha</u>         | <u>cliente</u> | valor |
|-----------------|---------------|----------------------|----------------|-------|
| Nescafe         | Providencia   | 2020-03-31T08:47:59Z | 413            | 3500  |
| Nescafe         | Providencia   | 2020-03-31T08:48:00Z | 413            | 3500  |





¿Preguntas?